

Ministry of Education and Research
"OVIDIUS" University of Constanța
Faculty of Mathematics and Computer Science
Major: Computer Science

Real-Time Detection and Prevention of Fraudulent Websites Using Machine Learning

Dissertation

Scientific Supervisor:

Lect. univ.dr Iordache Dorin

Graduate:

Dragomir Alin - Ciprian

Constanța

2026

Abstract

This master's thesis investigates the detection of phishing websites in a context where social engineering attacks are becoming increasingly sophisticated. The rapid development of Artificial Intelligence enables attackers to create convincing fraudulent content, making phishing attempts harder for users to recognize. Therefore, distinguishing between legitimate and malicious websites has become a major challenge, while traditional blacklist-based security methods are no longer sufficient.

The research proposes a real-time detection and prevention methodology based on Machine Learning techniques. The proposed system analyzes potential threats by examining the lexical and structural characteristics of URLs, together with the semantic content of web pages. This approach supports the identification of phishing attempts even when malicious websites imitate legitimate domains or hide fraudulent content within apparently trustworthy pages.

The result of this work is a software application designed to monitor web traffic and help users identify potentially dangerous resources. The system provides automated warnings and can prevent access to suspicious websites before sensitive information is exposed. The experimental evaluation shows improved detection capabilities compared to conventional blacklist-based approaches, contributing to better protection for users without advanced cybersecurity knowledge.

Rezumat

Această lucrare de disertație analizează detectarea site-urilor de tip phishing într-un context în care atacurile bazate pe inginerie socială devin tot mai sofisticate. Dezvoltarea rapidă a tehnologiilor de Inteligență Artificială le permite atacatorilor să creeze conținut fraudulos convingător, ceea ce face ca tentativele de phishing să fie mai dificil de recunoscut. Astfel, diferențierea dintre site-urile legitime și cele malițioase devine o provocare importantă, iar metodele tradiționale bazate pe liste negre nu mai sunt suficiente.

Cercetarea propune o metodologie de detectare și prevenire în timp real, bazată pe tehnici de Machine Learning. Sistemul propus analizează potențialele amenințări prin examinarea caracteristicilor lexicale și structurale ale URL-urilor, precum și a conținutului semantic al paginilor web. Această abordare permite identificarea tentativelor de phishing chiar și atunci când site-urile malițioase imită domenii legitime sau ascund conținut fraudulos în pagini aparent de încredere.

Rezultatul lucrării constă în dezvoltarea unei aplicații software pentru monitorizarea traficului web și sprijinirea utilizatorilor în identificarea resurselor periculoase. Sistemul oferă avertizări automate și poate preveni accesarea site-urilor suspecte înainte ca informațiile sensibile să fie expuse. Evaluarea experimentală arată o capacitate de detecție îmbunătățită față de metodele convenționale bazate pe liste negre, contribuind la protecția utilizatorilor fără cunoștințe avansate de securitate cibernetică.

Contents

Contents	ii
List of Figures	v
1 Introduction	2
1.1 Background and Motivation	2
1.2 Phishing Attacks and Emerging Threats	3
1.3 AI-Enhanced Cyber Attacks	4
1.4 Impact and Current State of Phishing Attacks	6
1.5 Objectives and Contributions of the Thesis	9
1.5.1 System Overview	9
1.5.2 Feature Engineering	9
1.5.3 Proxy-Based Traffic Iterception	10
1.6 Structure of the Thesis	11
2 Related Work	12
2.1 Traditional Phishing Detection Approaches	12
2.2 Machine Learning Based Detection Techniques	13
2.3 Recent AI-Based Security Solutions	15

2.4	Limitations of Existing Approaches	16
2.5	Research Gap	16
3	Proposed Solution	17
3.1	Overview of the Proposed Approach	17
3.2	System Architecture	18
3.3	Detection Methodology	20
3.4	Feature Extraction	22
3.5	Advantages of the Proposed Solution	24
3.6	Comparison with Existing Approaches	25
4	System Implementation and Evaluation	26
4.1	Application Architecture	26
4.2	Database Design	27
4.3	ML Model and Training Process	28
4.3.1	Random Forest Model	29
4.3.2	NLP-Based Model (URLBERT)	29
4.3.3	Training Observations	31
4.3.4	Backend Architecture and Bussines Logic	31
4.4	Python Model Server	31
4.5	Worker Component	32
4.6	ASP.NET Backend Server	33
4.7	Frontend Implementation	35
4.8	User Interaction and System Workflow	39
4.9	Performance Evaluation and Results	44
5	Conclusion and Future Work	46
5.1	Summary of the Research	46
5.2	Key Findings	46

5.3	Limitations of the Proposed Approach	47
5.4	Future Research Directions	47
	Bibliographic References	48

List of Figures

1.1	Comparison between a legitimate email and a clone phishing email illustrating how attackers replicate trusted communication while introducing subtle domain differences	4
1.2	Comparison of phishing attack detections between 2021 and 2022, highlighting the growth in daily detected malicious URLs and total annual phishing attacks .	6
3.1	Proposed system architecture for real-time phishing detection. The diagram shows the interaction between the MAUI frontend, ASP.NET backend, worker proxy component, Machine Learning model (NLP), and the local database. SignalR is used for real-time communication between components.	19
4.1	Application architecture of the proposed phishing detection system. The diagram illustrates the main components, including the MAUI frontend, ASP.NET backend, worker component, and the local database (LiteDB), as well as the flow of data between them.	27
4.2	Database structure showing the relationships between User, Device, ThreatEvent, and Alert entities.	28
4.3	Confusion matrix and classification report for the Random Forest model.	29
4.4	Training process of the NLP-based phishing detection model with 15 Epoch. . .	30
4.5	Training process of the NLP-based phishing detection model with 25 Epoch. . .	30
4.6	Login interface used to initialize the local user profile.	39
4.7	Dashboard showing aggregated threat statistics, monthly threat distribution, and current device information.	40
4.8	Live Requests page displaying intercepted traffic in real time.	41

4.9 Threat management page with filtering and allow/block actions. 41

4.10 Alert popup showing generated security notifications and alert management ac-
tions. 42

4.11 Local block page displayed when a detected phishing request is prevented. . . . 43

4.12 Local Windows proxy configuration used by the worker component during testing. 44

Chapter 1

Introduction

1.1 Background and Motivation

In recent years, the rapid development of Machine Learning (ML) and Natural Language Processing (NLP), often grouped under the broader concept of Artificial Intelligence (AI), has had a strong impact on the cybersecurity landscape. One of the areas where this impact is particularly visible is phishing. Compared to older forms of attacks, phishing campaigns have become more frequent, more convincing, and significantly harder to detect.

Traditionally, phishing attempts were relatively easy to identify due to obvious indicators such as poor grammar, suspicious links, or generic messages. However, this situation has changed. With the help of AI-based tools, attackers are now able to generate well-structured, context-aware messages that closely resemble legitimate communication. As a result, even users who are familiar with basic security practices can struggle to differentiate between genuine and malicious content.

Another important aspect is the expansion of the digital environment. Today, almost every individual owns at least one device connected to the internet, whether it is a smartphone, laptop, or IoT device. This widespread connectivity increases exposure to cyber threats and makes phishing attacks more accessible to a wider range of potential victims. While in the past less experienced users were considered the primary targets, recent trends suggest that vulnerability is no longer limited to a specific group. Even individuals with technical knowledge may fall victim to carefully crafted attacks, especially when these are powered by AI techniques such as text generation or personalization.

Considering these factors, there is a clear need for solutions that go beyond traditional rule-based detection methods. Modern threats require adaptive approaches capable of analyzing not only technical indicators, but also the content and context of communication. This motivated the idea behind the present work: developing a practical tool that leverages Machine Learning techniques to assist users in identifying potential phishing threats in real time.

Overall, the motivation for this work comes from the growing complexity of phishing attacks and the limitations of traditional defense mechanisms. By using Machine Learning in a defensive context, this research aims to explore how the same technologies that enable more advanced attacks can also be used to counter them effectively.

1.2 Phishing Attacks and Emerging Threats

Phishing has gradually evolved into one of the most significant threats in the digital environment, affecting both individuals and organizations. What makes it particularly dangerous is the combination of technical manipulation and human exploitation. Unlike purely technical attacks, phishing often relies on deceiving users, which makes it harder to defend against using traditional security mechanisms alone.

The concept of phishing dates back to the mid-1990s, when attackers first targeted America Online (AOL) users through simple social engineering techniques. At that time, the attacks were relatively basic, usually involving fake messages designed to trick users into sharing account credentials. However, over the years, phishing has developed into a much more complex and adaptable threat. Today, it spans multiple communication channels, including email, SMS, social media, and even voice calls, all used with the same goal: convincing victims to disclose sensitive information or interact with malicious content [1].

One key reason for this evolution is the increasing availability of tools that allow attackers to scale and automate their campaigns. Phishing is no longer limited to random messages sent in bulk. Instead, cybercriminals are able to design more refined strategies that target specific users or organizations. In many cases, the success of such attacks depends less on technical vulnerabilities and more on manipulating user behavior. In its simplest form, phishing can be described as a combination of social engineering and technical exploitation. Attackers often impersonate trusted entities such as banks, payment platforms, or well-known online services. To strengthen the illusion, they create fake websites or communication interfaces that closely resemble legitimate ones. This level of imitation has improved significantly over time, making it increasingly difficult for users to identify fraudulent content. The scale of the problem is also reflected in statistical data. For example, reports from the Anti-Phishing Working Group (APWG) have shown a steady increase in phishing activity over the years, with hundreds of thousands of new threats detected on a daily basis during peak periods. Such trends highlight not only the persistence of phishing as a threat, but also the need for more effective detection and prevention mechanisms.

Types of Phishing Attacks

Phishing attacks have diversified into several categories, each with its own characteristics and level of sophistication. Understanding these variations is important, as it helps in recognizing potential threats more effectively.

One of the most common forms is standard phishing. This approach typically involves sending generic messages to a large number of users, pretending to come from trusted organizations. Although less personalized, this method remains effective due to its scale and low

cost. A more advanced variant is spear phishing, where the attacker targets specific individuals or organizations. In this case, the message is tailored using available information about the victim. This personalization increases credibility and makes the attack harder to detect.

Another interesting technique is clone phishing. Instead of creating a completely new message, attackers replicate legitimate emails and modify certain elements, such as links or attachments. Because the structure appears familiar, recipients are more likely to trust the content without noticing the subtle changes. Phishing also extends beyond email communication. Messaging-based attacks have become increasingly common, especially with the widespread use of mobile devices. These include smishing, where fraudulent messages are delivered via SMS, and vishing, where attackers use phone calls to impersonate trusted entities. Both methods rely heavily on urgency and psychological pressure to influence user decisions.

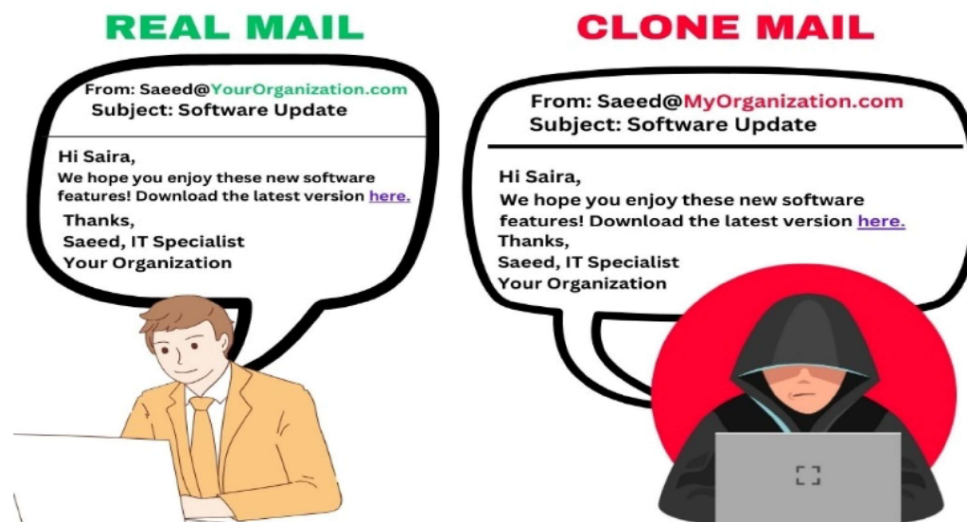


Figure 1.1: Comparison between a legitimate email and a clone phishing email illustrating how attackers replicate trusted communication while introducing subtle domain differences

In the context of web-based attacks, techniques such as pharming and search engine manipulation play an important role. Pharming involves redirecting users to fake websites by altering DNS settings, often without their knowledge. On the other hand, search engine phishing relies on placing malicious websites in search results, taking advantage of user trust in popular platforms. These methods operate at a different level compared to traditional phishing, making detection more complex [2].

1.3 AI-Enhanced Cyber Attacks

The integration of Artificial Intelligence (AI) into cybersecurity has introduced both new defensive opportunities and new forms of threats. While significant attention has been given to the ability of Machine Learning models to detect phishing attempts, it is equally important to understand how the same technologies are being used by attackers to improve their

methods. In this context, phishing campaigns have undergone a noticeable transformation. Although AI-based systems can assist in identifying malicious content, they are not without limitations. Their effectiveness depends heavily on the quality and representativeness of the data used during training. In practice, this means that biased or incomplete datasets can lead to incorrect predictions, either by missing real threats or by incorrectly flagging legitimate communication. At the same time, attackers are constantly adapting, developing strategies specifically designed to bypass automated detection systems. For this reason, any AI-based defense approach must be continuously updated and refined in order to remain effective [1].

Impact of AI on the Phishing Lifecycle

The introduction of AI into phishing campaigns has significantly altered the traditional lifecycle of such attacks. Previously, phishing followed a relatively straightforward sequence: target identification, message creation, delivery, and exploitation. Today, these stages are increasingly automated and optimized through AI techniques. One of the most noticeable changes is the level of efficiency. AI allows attackers to automate processes that previously required manual effort, making campaigns faster and easier to scale. As a result, phishing is no longer limited by human resources in the same way as before. At the same time, this evolution creates additional challenges for real-time detection systems, which must now deal with attacks that are both faster and more adaptive.

Automation of Phishing Campaigns

AI has significantly reduced the amount of manual work required to conduct phishing campaigns. For example, attackers can automatically collect data from publicly available sources such as social media profiles or online platforms. This information can then be used to tailor messages to specific users. Because of this automation, phishing campaigns can reach a much larger number of potential victims with relatively little effort. At the same time, the messages generated tend to feel more relevant and personalized, which increases the likelihood of user interaction. In practice, this makes traditional large-scale filtering less effective, as attacks no longer follow a uniform pattern.

Improved Generation of Phishing Content

Another important aspect is the ability of AI systems to generate content that closely resembles legitimate communication. By learning from large datasets containing real emails, websites, or messages, AI models can replicate writing styles, structure, and even visual elements. This means that phishing emails are no longer easy to identify based on obvious mistakes. In many cases, they are written in a clear and professional way, without the typical errors that used to serve as warning signs. As a result, detection methods that rely solely on superficial differences between legitimate and malicious content are becoming less reliable [1].

Adaptive and Evolving Attack Strategies

A key feature of AI-based systems is their ability to learn from previous outcomes. This applies not only to defensive systems, but also to offensive tools used in phishing campaigns. Attackers can analyze which messages were successful and adjust their strategies

accordingly. Over time, this creates a cycle of continuous improvement. If a certain phishing format proves effective, it can be reused or slightly modified in future attempts. This adaptability makes attacks more resilient to standard detection techniques, as the patterns that systems rely on are constantly changing.

Real-Time Execution of Phishing Attacks

AI also enables the rapid generation and delivery of phishing content. Instead of preparing messages in advance, attackers can create and deploy them almost in real time. This allows campaigns to react quickly to new opportunities, such as trending topics or ongoing events. From a defensive perspective, this creates additional pressure on detection systems. It is no longer enough to analyze threats retrospectively; systems must be capable of identifying and blocking attacks as they occur. This requirement significantly increases the complexity of real-time threat detection.

1.4 Impact and Current State of Phishing Attacks

The growing scale of phishing attacks has led to increased interest in automated detection methods. Machine Learning, in particular, has shown strong potential in identifying malicious URLs and suspicious communication patterns. Recent reports indicate a steady increase in phishing activity. For example, in 2021, approximately 50,000 malicious URLs were detected daily. This number increased significantly in 2022, reaching around 80,000 per day, which represents a growth of over 60 percent. In total, this corresponds to hundreds of millions of phishing attempts identified annually using AI-based detection systems.

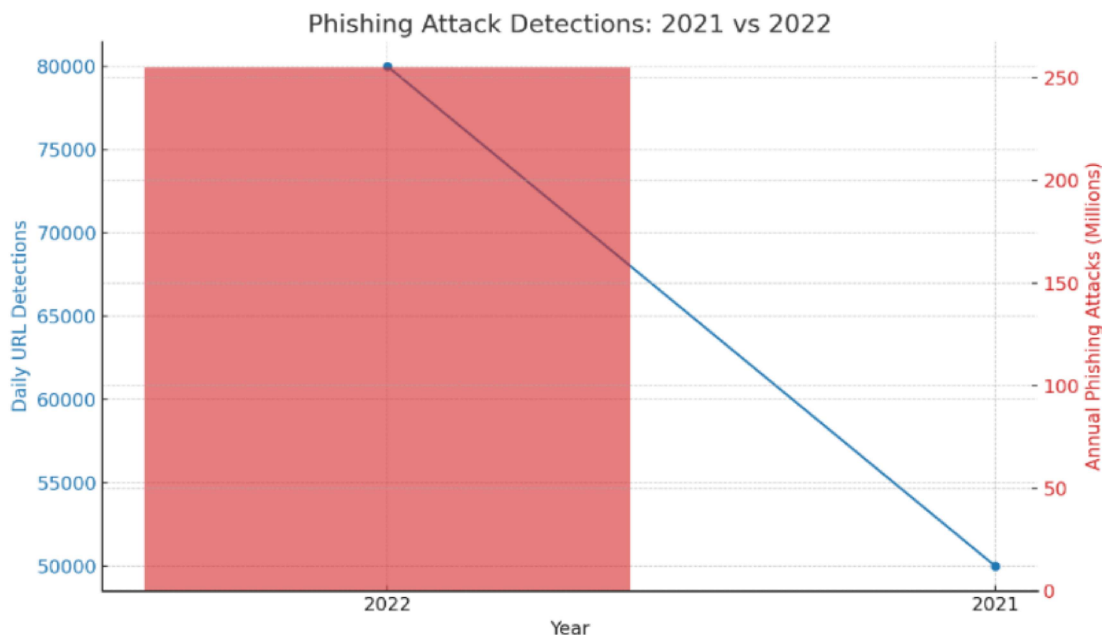


Figure 1.2: Comparison of phishing attack detections between 2021 and 2022, highlighting the growth in daily detected malicious URLs and total annual phishing attacks

The evolution of phishing attacks in recent years is illustrated in Figure 1.2. This figure highlights both the scale of the problem and the importance of developing effective detection mechanisms. At the same time, it suggests that attackers are continuously adapting their techniques, which requires equally dynamic and intelligent defensive solutions [1].

One of the defining characteristics of modern phishing is its ability to integrate emerging technologies. In particular, the use of Artificial Intelligence has significantly transformed how these attacks are designed, optimized, and executed. As a result, traditional assumptions about the appearance and behavior of phishing attempts are no longer always valid.

AI-Driven Phishing

AI-driven phishing has changed the way attacks are created and delivered. One important capability is the analysis of user behavior. AI-based tools can study how users normally interact with services, for example when they are most active or how they respond to notifications. Based on this information, attackers can choose the best moment to send phishing messages, increasing the chances that users will interact with them.

Another important aspect is the ability to personalize attacks. Instead of sending the same message to many users, attackers can create messages that look more relevant to a specific person or organization. For example, they can imitate internal communication styles or copy the way employees usually talk to each other.

This level of personalization makes phishing messages appear more natural and trustworthy. As a result, it becomes much harder for users to tell the difference between a real message and a malicious one. Traditional detection methods, which often rely on simple patterns, may not be enough to detect these types of attacks.

Deepfake Phishing

Deepfake phishing is a newer type of attack that uses artificial intelligence to create fake audio or video content. These recordings can sound or look very similar to real people, which makes them very convincing.

In a typical case, an attacker may generate a voice recording that sounds like a manager or an executive from a company. This recording can then be used in a phone call to convince someone to take an action, such as sending money or sharing confidential information.

The main problem is that these recordings are difficult to verify. In many organizations, especially those that use remote communication, people often trust voice calls or video messages. Because of this, deepfake phishing can be very effective.

This type of attack shows that checking identity is becoming more difficult. Systems that rely only on voice or visual confirmation are no longer enough, and new ways of verification are needed.

Mobile Phishing and Smishing

The use of mobile devices has created new opportunities for phishing attacks, especially through SMS messages, also known as smishing. Compared to desktop environments, mo-

mobile devices often show less information on the screen. This makes it harder for users to check things like full URLs or security details.

Another important factor is user behavior. People tend to react quickly to notifications on mobile devices without thinking too much about the content. This increases the chance that they will click on a malicious link or follow instructions from a phishing message.

Mobile devices are also used for many daily activities, such as banking or communication, which makes them more attractive targets for attackers. Because of this, phishing attacks on mobile platforms are becoming more common and more successful.

IoT-Based Phishing

The expansion of the Internet of Things (IoT) has introduced additional vulnerabilities that can be exploited in phishing-related attacks. Many IoT devices are designed with limited security features and are often not updated regularly, which makes them easier targets for attackers.

Another important issue is that these devices usually operate in the background, without direct user interaction. Because of this, users are often not aware when something suspicious happens. This increases the risk that compromised devices remain undetected for a long time.

Once exploited, these devices can serve as entry points into larger networks, allowing attackers to bypass traditional security layers. For example, an attacker can use a vulnerable smart device to gain access to other systems connected to the same network. In some situations, compromised IoT devices can also be used as part of larger attack infrastructures, supporting phishing campaigns or spreading malicious content.

The growing number of connected devices makes this problem even more serious. Modern environments often include smartphones, smart home systems, and other connected devices that communicate with each other. This means that a single weak point can affect multiple systems at the same time.

For these reasons, IoT environments should be considered when designing modern cybersecurity solutions. Protecting only traditional systems is no longer enough, as attackers increasingly look for indirect ways to reach their targets.

Conclusion of the Section

These developments highlight the limitations of traditional detection methods and emphasize the need for adaptive, context-aware security solutions. Understanding how phishing evolves is essential for designing systems that can respond effectively, especially in real-time scenarios.

1.5 Objectives and Contributions of the Thesis

The purpose of this dissertation is not limited to analyzing phishing attacks, but also to provide a practical solution that addresses some of the challenges identified in previous chapters. The main contribution of this work consists in the design and implementation of an application that combines Machine Learning techniques with a user-oriented approach, focusing both on detection accuracy and usability.

1.5.1 System Overview

One of the key aspects considered during development was accessibility. Many existing cybersecurity tools assume a certain level of technical knowledge, which limits their effectiveness in real-world scenarios. In this project, a different approach was taken. The application was built using .NET MAUI, allowing the creation of a modern and flexible user interface that remains simple and easy to use. The interface was intentionally designed to minimize complexity, so that even non-technical users can understand and interact with the system without difficulty. This decision reflects the idea that protection against phishing should not be limited only to experienced users. From an architectural perspective, the solution includes a background component implemented as a worker service, which acts similarly to a proxy server. Once the application starts, this component is automatically configured and begins intercepting outgoing network requests. This approach allows the system to analyze accessed URLs in real time, before the user interacts with potentially malicious content. Instead of relying only on static checks, the application integrates a Machine Learning model capable of evaluating each request dynamically.

1.5.2 Feature Engineering

Another important part of the contribution lies in the dataset preparation process. Rather than relying on a single data source, multiple datasets were combined in order to improve diversity and overall quality. These datasets, obtained from publicly available repositories, contain information related to URLs and phishing behavior. However, since they differ in format and structure, a custom preprocessing pipeline was developed.

Several functions were implemented specifically for this purpose. For example, URL feature extraction was handled through a custom processing step that identifies relevant attributes such as length, domain structure, number of special characters, and other patterns commonly associated with malicious links. Additional transformations were applied to standardize the data and convert it into a format suitable for training Machine Learning models. At the same time, different labeling schemes present in the original datasets were unified into a simpler binary classification: Safe or Threat. To better explore the problem, two different types of datasets were created. The first one is a structured, tabular dataset containing engineered numerical features extracted from URLs. This version is suitable for classical

Machine Learning approaches, where features need to be explicitly defined. The second dataset follows a different strategy. Instead of extracting features manually, it preserves the raw URL strings and allows the model to learn patterns directly from text. This dataset was used to train a classification model, which can identify more subtle patterns that may not be visible through simple feature engineering.

Based on these datasets, two distinct approaches were implemented and evaluated. The first approach uses a classical Machine Learning model, specifically a Random Forest classifier. This model was selected due to its robustness and its ability to handle structured data effectively. It operates on the engineered features extracted from URLs and provides reliable performance without requiring complex preprocessing steps. The second approach is based on a transformer model, URL BERT, which operates directly on textual input. Instead of manually defining features, this model learns representations automatically, using patterns found in the URL strings themselves. The model was further improved through fine-tuning, using the dataset created during the preprocessing stage. This step allows the model to adapt to the specific characteristics of the phishing data used in this project.

An additional contribution is the integration layer between the model and the user interface. A backend component was developed to manage communication and ensure real-time interaction between different parts of the system. For this purpose, SignalR was used, enabling efficient transmission of data between the worker service and the application interface. This allows users to receive immediate feedback when a potential threat is detected.

1.5.3 Proxy-Based Traffic Iterception

In practical terms, when a user attempts to access a website, the request is intercepted by the proxy component and analyzed using the selected model. Based on the prediction result, the application can block or allow access, while also notifying the user. Importantly, the system does not completely remove user control. Instead, it provides the necessary information and allows the user to decide whether to proceed.

Finally, a comparative analysis was conducted between the two modeling approaches. This comparison helps highlight the strengths and limitations of both classical Machine Learning techniques and transformer-based models in the context of phishing detection. Beyond performance metrics, this evaluation also considers practical aspects, such as response time and integration feasibility.

Overall, the contribution of this work lies in combining multiple elements into a single, functional system: dataset engineering, model training, real-time detection, and user interaction. Rather than focusing exclusively on theoretical performance, the project aims to demonstrate how Machine Learning can be applied in a practical scenario, offering a solution that remains both effective and usable.

1.6 Structure of the Thesis

This thesis is organized into five main chapters, each addressing a specific part of the research process, starting from the general context and gradually moving toward the proposed solution and its evaluation.

The first chapter introduces the overall context of the study. It presents the background and motivation, focusing on the increasing relevance of phishing attacks in the modern digital environment. In addition, it explores the evolution of phishing techniques, highlighting emerging threats and the role of Artificial Intelligence in both enabling and detecting such attacks. The chapter also discusses the current impact of phishing and outlines the main objectives and contributions of the thesis.

The second chapter focuses on related work and provides an overview of existing approaches in phishing detection. It begins with traditional techniques based on rule-based systems and heuristics, followed by more recent developments involving Machine Learning models. The chapter also reviews modern AI-based security solutions and highlights their limitations. Based on this analysis, a research gap is identified, which motivates the development of the proposed solution.

The third chapter presents the proposed solution. It describes the overall approach and introduces the system architecture. The detection methodology is explained in detail, including the process of feature extraction and the use of Machine Learning techniques. The chapter also discusses the advantages of the proposed approach and compares it with existing solutions, emphasizing the improvements introduced.

The fourth chapter focuses on the implementation and evaluation of the system. It provides a detailed description of the application architecture, including the database design, backend logic, and frontend components. The Machine Learning model and the training process are also presented. In addition, the chapter explains how users interact with the system and how requests are processed in real time. Finally, the experimental setup is described, followed by the evaluation of the system's performance and the obtained results.

The final chapter summarizes the main findings of the research. It highlights the most important contributions of the work and discusses its limitations. The chapter concludes with several directions for future research, suggesting possible improvements and extensions of the proposed system.

Chapter 2

Related Work

The domain of phishing detection has evolved significantly over the past decade, driven by the rapid advancement of both cyberattack techniques and defensive technologies. With the increasing integration of Artificial Intelligence into both attack and defense mechanisms, phishing has become more adaptable, scalable, and difficult to detect. As a result, researchers have explored a wide spectrum of detection approaches, ranging from traditional rule-based mechanisms to advanced deep learning architectures.

Recent studies emphasize that phishing is no longer limited to a single attack vector, but instead combines multiple techniques such as social engineering, content manipulation, and platform-specific exploitation. This complexity requires detection systems that are not only accurate, but also adaptive and capable of handling diverse data sources.

2.1 Traditional Phishing Detection Approaches

Phishing detection methods were primarily based on rule-based and heuristic techniques. These approaches focus on identifying predefined patterns such as blacklisted URLs, suspicious domain structures, or irregularities in email formatting. These methods rely on the assumption that malicious content can be detected by matching known signatures or fixed indicators, making them relatively simple to implement and computationally efficient.

One of the most widely used traditional techniques is blacklist-based detection. As highlighted by Alanezi, this approach has been adopted extensively due to its simplicity and low resource requirements, making it suitable for large-scale filtering systems and environments with limited computational power [3]. In practice, blacklists consist of collections of previously identified malicious domains or URLs, which are used to block access to known threats. However, this strategy introduces a significant limitation, as it can only detect attacks that have already been observed and recorded. Consequently, new or modified phishing attempts, commonly referred to as zero-day attacks, are able to bypass such defenses with relative ease.

Beyond blacklist techniques, traditional methods also include heuristic-based analysis, which attempts to identify suspicious behavior through predefined rules. These rules may include checking for abnormal URL length, excessive use of special characters, or inconsistencies in domain naming conventions. While these heuristics can improve detection rates compared to simple blacklists, they remain dependent on manually defined criteria. As attackers continuously adapt their strategies, maintaining and updating these rules becomes increasingly difficult, reducing their effectiveness over time.

Valiyaveedu et al. categorize traditional phishing detection methods into three main approaches: URL-based detection, HTML-based analysis, and visual similarity techniques [4]. URL-based detection focuses on analyzing structural properties of web addresses, such as domain composition, path complexity, and the presence of suspicious keywords. HTML-based analysis, on the other hand, examines the underlying code of web pages, looking for anomalies such as hidden elements, redirections, or malicious scripts embedded in the content. Visual similarity techniques attempt to detect phishing websites by comparing their appearance to legitimate sites, identifying cases where attackers replicate user interfaces in order to deceive users.

Each of these approaches contributes to improving detection, they all share common limitations. One of the main challenges is their inability to cope with the dynamic and rapidly evolving nature of phishing attacks. Modern phishing campaigns often involve subtle modifications that are sufficient to bypass rule-based systems while still appearing convincing to users. As noted by Lamina et al., attackers constantly refine their techniques, making traditional detection mechanisms increasingly difficult to maintain and less effective over time [5].

Another important limitation is the lack of adaptability. Traditional systems do not learn from new data unless explicitly updated, which creates a gap between emerging threats and detection capabilities. This reactive nature makes them unsuitable for environments where attacks evolve quickly and unpredictably. Furthermore, these methods often fail to capture contextual information, such as the intent behind a message or the relationship between different elements of a URL or web page.

Despite these shortcomings, traditional phishing detection approaches still play an important role in modern cybersecurity solutions. They are often used as a first line of defense, providing fast and efficient filtering of known threats. However, they are no longer sufficient as standalone solutions. Instead, they need to be complemented by more advanced techniques, such as Machine Learning and Artificial Intelligence, which can adapt to new attack patterns and provide more robust detection capabilities.

2.2 Machine Learning Based Detection Techniques

Machine Learning approaches have gradually become a preferred solution for phishing detection, mainly because traditional rule-based systems are no longer sufficient on their own. Instead of relying on predefined rules, ML models learn directly from data, which allows them to capture patterns associated with malicious behavior in a more flexible way.

These models analyze large datasets and look for relationships between input features and classification outcomes. Through this process, they can detect patterns that are not immediately visible to rule-based systems or manual analysis. This capability is particularly relevant in phishing detection, where attackers continuously adjust their techniques in order to avoid detection.

As described by Alanezi, ML models usually operate on features extracted from URLs, email content, and web pages [3]. These features often include structural properties such as URL length, the number of special characters, and domain-related attributes, but may also involve lexical or semantic characteristics derived from text. Combining these different types of information allows the model to build a more detailed representation of each instance.

A wide range of algorithms has been explored in this context. Decision Trees, Random Forest, Support Vector Machines, and ensemble methods are among the most commonly used techniques. Random Forest models, in particular, are frequently applied because they handle high-dimensional data well and tend to produce stable results. Ensemble approaches also play an important role, since combining multiple models can improve generalization and reduce sensitivity to noise in the data.

Studies such as the one conducted by Ahmad et al. show that ML-based solutions perform significantly better when trained on large and diverse datasets [6]. One of their strongest points is the ability to generalize, which means they can detect phishing attempts that were not explicitly seen during training. This characteristic makes them more effective than rule-based systems, which are limited to predefined patterns.

ML models have also been applied to more complex phishing scenarios. Atlam et al. discuss the role of Machine Learning in detecting Business Email Compromise attacks, where malicious intent is hidden behind otherwise legitimate communication [7]. In such cases, there may be no suspicious links or attachments, so detection relies more on analyzing context, writing style, and communication behavior.

Even with these advantages, some challenges remain. The performance of ML models is closely linked to the quality of the data used during training. If the dataset is incomplete or imbalanced, the model may produce unreliable results. Feature engineering can also influence the outcome, especially in cases where important characteristics are overlooked during preprocessing.

Another issue that needs attention is the balance between false positives and false negatives. As highlighted by Shahbazi et al., a high false positive rate can cause users to ignore warnings, while false negatives can allow malicious content to pass through undetected [8]. Finding a balance between these two aspects is essential for building a system that is both secure and usable.

There is also growing interest in how attackers may attempt to exploit Machine Learning systems. By carefully modifying inputs, it is possible to create adversarial examples that reduce model accuracy. This suggests that detection models cannot remain static and need to be updated regularly in order to remain effective.

In conclusion Machine Learning has clearly improved phishing detection capabilities,

offering better adaptability compared to traditional approaches. At the same time, these models require careful training, proper data handling, and continuous updates to remain reliable in a constantly changing threat landscape.

2.3 Recent AI-Based Security Solutions

In recent years, research in phishing detection has gradually shifted toward more advanced AI-driven approaches, especially those based on deep learning and Natural Language Processing. This change did not happen by chance. Traditional Machine Learning methods, while useful, often depend on manually defined features and are not flexible enough when dealing with constantly evolving threats.

One clear advantage of modern AI-based solutions is that they can work directly with raw input data. Instead of relying on predefined rules or extracted attributes, these models learn patterns on their own. This allows them to capture hidden relationships and subtle variations, which are very common in phishing attacks where small changes are used to avoid detection.

Catal et al. show that deep learning models such as Deep Neural Networks, Convolutional Neural Networks, and Long Short-Term Memory architectures can achieve strong results across different phishing detection scenarios [9]. What makes these models effective is their ability to process data in layers, gradually building a deeper understanding of patterns. This becomes particularly useful when working with complex inputs like URLs, web content, or email structures.

At the same time, NLP-based methods have become more relevant, especially when analyzing phishing emails. Egozi and Verma point out that linguistic features such as sentence structure, punctuation patterns, and word distribution can reveal important signals about malicious intent [10]. Instead of looking only for specific keywords, these approaches examine how the text is written, making them more resilient to simple obfuscation techniques.

Another noticeable direction in recent work is the use of combined analysis methods. Rather than focusing on a single type of data, newer systems try to bring together multiple sources of information. Lamina et al. highlight that combining user behavior, network activity, and textual analysis can lead to better detection performance [5]. By looking at both content and user interaction, such systems are able to detect more complex attack scenarios.

There is also interest in extending phishing detection beyond text-based inputs. For instance, Yao et al. explore the use of computer vision techniques for analyzing QR codes through logo recognition [11]. This shows that phishing is no longer limited to emails or websites, but is expanding into other forms of digital interaction.

At the same time, these solutions come with their own challenges. Deep learning models usually need large amounts of data and can be harder to train and maintain. Another issue is that their decision-making process is not always easy to explain, which can raise concerns when trying to understand why a certain input was classified as malicious.

Overall, AI-based detection methods have clearly improved the state of phishing detection. They provide better adaptability and can handle more complex attack patterns. At the same time, their successful use depends on careful design and continuous updates, especially as attackers continue to refine their techniques.

2.4 Limitations of Existing Approaches

Despite significant progress, several limitations remain in current phishing detection methods.

One of the primary challenges is the reliance on large, high-quality datasets. As emphasized by Ahmad et al., the performance of AI models is directly linked to the availability of labeled data, which is often difficult to obtain [6]. Additionally, many studies do not apply feature selection techniques, leading to inefficiencies and reduced model interpretability.

Another challenge is the adversarial nature of phishing attacks. Attackers continuously evolve their methods, often using AI themselves to bypass detection systems. Shahbazi et al. highlight that this creates an ongoing arms race between attackers and defenders [8].

In practice detection systems often produce false positives, which can lead users to ignore warnings, and false negatives, which allow attacks to succeed. Usability also remains a concern, as complex systems may not be suitable for non-technical users [5].

2.5 Research Gap

The literature reveals several important gaps that need to be addressed.

First, many studies focus primarily on improving model accuracy, while neglecting real-world usability. Detection systems must not only be accurate, but also easy to use and understand.

Second, there is limited emphasis on real-time detection at the user level. Most solutions operate in centralized environments, leaving individual users without direct protection.

Third, although AI models have shown strong performance, their integration into complete, deployable systems remains limited. As highlighted by Ahmad et al. and Catal et al., future research should focus on building adaptive systems that combine detection accuracy with real-time capabilities and user interaction [6, 9].

This dissertation addresses these challenges by proposing a system that integrates a transformer-based model into a real-time architecture, combined with a user-friendly interface and proxy-based traffic interception, providing both practical usability and effective threat detection.

Chapter 3

Proposed Solution

The solution combines several components, including a user interface, a backend service, a proxy-based worker, and a Machine Learning model for classification. These components work together to provide continuous protection, while still allowing the user to remain in control of the final decision.

The following sections describe the proposed approach in detail, including the system architecture, detection methodology, and feature extraction process.

3.1 Overview of the Proposed Approach

The starting point of this work was a simple question: how can phishing detection be improved in a way that is not only accurate, but also practical for everyday users? While many existing solutions focus on detection performance, fewer consider how such systems are actually used in real-world scenarios. This observation led to the idea of developing a system that combines Machine Learning with a user-friendly interface, making phishing protection more accessible.

The initial concept was to build a solution that focuses on URL-based detection, as URLs are often the first interaction point in phishing attacks. Instead of analyzing threats after they have already affected the system, the goal was to detect potential risks in real time, before the user accesses a malicious resource. The development process started with defining the overall structure of the application. This included designing a database model suitable for storing relevant information, such as analyzed URLs, detection results, and user interactions. A backend layer was then implemented, providing the necessary APIs and CRUD functionality required for communication between system components.

In parallel, attention was given to the frontend part of the application. The goal here was not to create a complex interface, but rather a simple and intuitive environment where users can easily understand what is happening and make quick decisions. This is particularly important, considering that not all users have technical expertise. A key challenge was inte-

grating the Machine Learning model into a real-time environment. It was necessary to find a way to capture user requests and evaluate them without interrupting normal system behavior. To achieve this, a worker service was introduced. This component runs continuously in the background, as long as the system is active, and acts as an intermediary between the user interface and the backend logic.

The worker behaves similarly to a proxy, intercepting outgoing URL requests. Once a request is captured, it is passed to the Machine Learning model for analysis. For this purpose, a transformer-based model (BERT architecture) was used and fine-tuned on a custom dataset prepared specifically for this project. This allows the system to classify URLs in real time as either safe or potentially malicious. Since the system needs to provide feedback instantly, even when the user interface is not actively open, real-time communication becomes essential. To address this, SignalR was integrated into the architecture. This enables live updates between the worker, backend, and frontend, ensuring that the user is immediately notified when a threat is detected.

As a result, the system operates continuously, analyzing traffic in the background and providing notifications when necessary. The user is informed about potential risks and can decide whether to proceed or block access. This approach creates a balance between automated detection and user control. The proposed approach combines several elements—real-time traffic interception, Machine Learning-based analysis, and user-oriented design—into a single system. The focus is not only on detecting phishing attacks, but also on delivering a solution that can be realistically used in everyday situations.

3.2 System Architecture

The architecture of the proposed system was designed with the intention of ensuring real-time phishing detection while maintaining a simple and accessible user experience. Instead of relying on a single component, the system integrates multiple elements that work together in a coordinated manner. These include the frontend application, backend services, a background worker acting as a proxy, and a Machine Learning model used for classification. At a high level, the system follows a modular structure. Each component has a clearly defined role, and communication between them is handled in real time. This design allows the system to remain both flexible and efficient, while also supporting continuous monitoring of user activity.

Frontend Layer (MAUI Application)

The user interface was developed using .NET MAUI, which enables cross-platform compatibility and a modern application design. The main goal of this component is to provide a simple and intuitive interaction model. The interface is responsible for displaying detection results, alerts, and system status updates. When a potentially malicious URL is detected, the user is notified immediately and can take action, such as allowing or blocking access. The design avoids unnecessary complexity, making the application usable even for individuals without technical expertise [12].

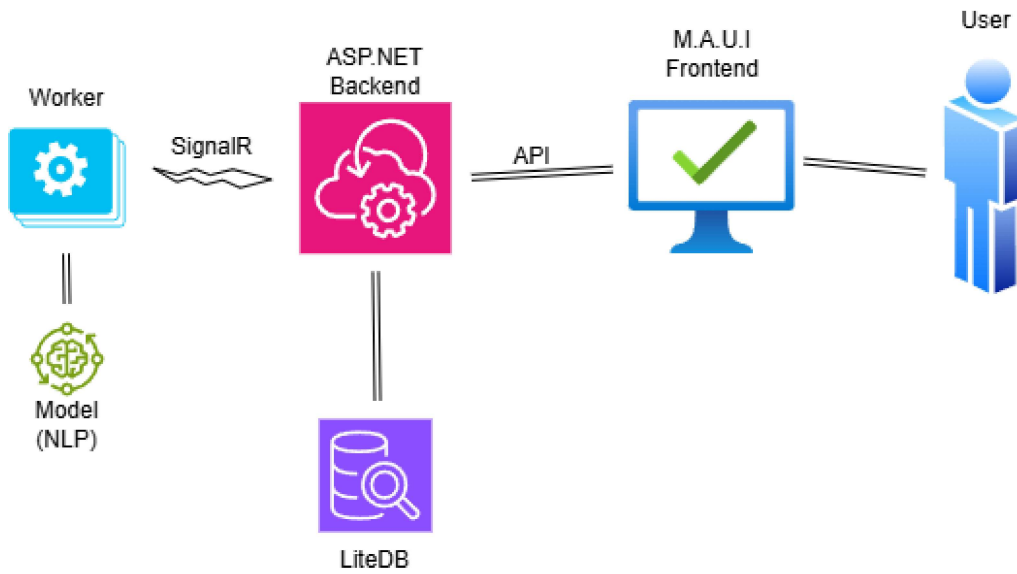


Figure 3.1: Proposed system architecture for real-time phishing detection. The diagram shows the interaction between the MAUI frontend, ASP.NET backend, worker proxy component, Machine Learning model (NLP), and the local database. SignalR is used for real-time communication between components.

Worker Service (Proxy and Traffic Monitoring)

A central element of the architecture is the worker service, implemented in C#. This component operates continuously in the background and is automatically started when the application is launched. The worker acts as a proxy, intercepting outgoing web requests before they reach their destination. This allows the system to analyze traffic in real time, rather than reacting after the fact. Every intercepted URL is processed and forwarded to the Machine Learning model for evaluation. In addition to traffic monitoring, the worker also manages communication with both the backend and the frontend. It sends analyzed data to the backend for storage and, at the same time, triggers real-time notifications that are displayed to the user. One important aspect is that the worker remains active even when the user interface is not open. This ensures continuous protection, as detection is not dependent on user interaction.

Machine Learning Integration

The core detection mechanism is based on a Machine Learning model trained to classify URLs as either safe or malicious. The model is integrated directly into the system and is used by the worker service for real-time evaluation. A transformer-based architecture (BERT) was selected and fine-tuned using a custom dataset. This allows the model to identify patterns directly from the structure of URLs, rather than relying only on predefined rules. When a URL is intercepted by the worker, it is passed to the model, which returns a classification result. This result is then used to decide whether the request should be considered safe or a potential threat [13].

Backend Layer (ASP.NET and SQLite)

The backend of the system was implemented using ASP.NET [12], providing a structured environment for managing application logic and data. Its main responsibilities include:

- ▷ Storing analyzed URLs and classification results
- ▷ Managing communication between system components
- ▷ Exposing APIs used by the worker and frontend

For data persistence, SQLite was used as the database solution. This allows all data to be stored locally on the user's device, which improves performance and reduces dependency on external services. At the same time, it helps maintain user privacy, as no sensitive data needs to be transmitted externally.

Real-Time Communication (SignalR)

To ensure seamless interaction between components, SignalR was integrated into the system. This technology enables real-time communication between the worker, backend, and frontend. When the worker processes a URL and obtains a result from the Machine Learning model, the information is immediately sent to the frontend using SignalR.

This allows the user to receive instant notifications, without the need for manual refresh or periodic polling. This real-time aspect is essential for phishing detection, as it enables immediate response to potential threats.

System Workflow

The overall system follows a continuous workflow:

- ▷ The user initiates a web request (accessing a URL)
- ▷ The worker service intercepts the request and acts as a proxy
- ▷ The intercepted URL is passed to the Machine Learning model
- ▷ The model analyzes the URL and returns a classification result
- ▷ The result is sent to the backend for storage
- ▷ Through SignalR, the frontend is updated in real time
- ▷ The user is notified and can decide whether to proceed or block access

This workflow ensures that phishing detection occurs before the user interacts with potentially malicious content.

3.3 Detection Methodology

The detection methodology adopted in this work is centered around the idea that phishing attacks, especially those targeting URLs, can be effectively approached as a textual classification problem. Instead of relying on traditional feature extraction techniques, where specific characteristics such as URL length or the number of special characters are manually defined, this approach allows the model to analyze the entire structure of a URL as a sequence of text. In this way, the system benefits from a more flexible and expressive representation, capable of capturing patterns that might not be immediately visible through manual analysis.

The core of this methodology is built around the transformer-based model 'CrabInHoney/urlbert-tiny-v4-phishing-classifier' [13]. This model was chosen due to its ability to capture complex contextual relationships between tokens, an advantage that becomes particularly relevant in phishing detection. URLs are often designed to appear legitimate at first glance, but subtle differences in structure, ordering, or character usage can indicate malicious intent. A model such as URLBERT is able to analyze these relationships in a more nuanced way, going beyond surface-level patterns and identifying combinations of elements that would otherwise be difficult to interpret using conventional methods.

In order to apply this model to the phishing detection problem, an adaptation step is required. Although URLBERT is pre-trained on general language tasks, it does not initially have knowledge specific to URLs or cybersecurity threats. To address this, the model is fine-tuned using a dataset specifically prepared for this purpose. The dataset consists of labeled examples where each URL is associated with a classification, either representing a safe resource or a potential threat. Through this process, the model gradually learns to distinguish between legitimate and malicious patterns, adjusting its internal parameters based on the characteristics present in the training data.

An important aspect of this methodology is the decision to work directly with raw input. Instead of simplifying the problem into predefined attributes, the full structure of each URL is preserved. This allows the model to identify patterns related to domain composition, subdomain usage, uncommon strings, or obfuscation techniques. In practice, phishing URLs often rely on very small manipulations, such as inserting additional characters, using visually similar words, or reorganizing parts of a legitimate domain name. These variations are difficult to capture through simple rules, but can be learned naturally by a transformer-based model.

The integration of the model into the system is designed to support real-time analysis. Whenever a user attempts to access a website, the request is intercepted by the worker service, which acts as a proxy between the client and the external network. At this stage, the URL is extracted and passed to the URLBERT model for classification. The inference process is performed immediately, without introducing significant delay to the user experience. This is an important requirement, as detection must occur before the user interacts with the content.

Once the model produces a classification result, the system uses this information to determine how the request should be handled. If the URL is considered safe, the request is allowed to proceed normally. In contrast, if the model identifies the URL as a potential threat, the system reacts by blocking the request automatically. This behavior reflects a defensive strategy where protection is applied by default, reducing the likelihood of user exposure to malicious content.

At the same time, the system does not operate as a fully restrictive mechanism. After a detection is made, the user is notified in real time through the application interface. This notification includes the classification result and allows the user to decide whether to proceed despite the warning. In practice, this approach maintains a balance between automation and user control. The system provides immediate protection, but still respects the possibility that a classification may not always be perfect, especially in borderline cases.

This combination allows the system to function as a proactive defense mechanism, rather than a reactive tool, which is essential in the context of modern cybersecurity threats.

3.4 Feature Extraction

Feature extraction and dataset preparation were performed using multiple data sources in order to improve the quality and diversity of the training data. Instead of relying on a single dataset, three different public datasets were selected from Kaggle. Each of these datasets contained URL-related data, but they were structured in different ways and used different labeling formats.

In this project, feature extraction was performed using two different approaches, in order to evaluate both traditional Machine Learning methods and modern NLP-based models.

The first approach focuses on transforming URLs into structured numerical features. This method is designed to be used with classical Machine Learning algorithms such as Random Forest. In this case, each URL is processed and converted into a set of attributes that describe its structure.

To achieve this, a custom function was implemented to extract features from the URL. These features include the total length of the URL, the number of dots and slashes, the type of protocol (HTTP or HTTPS), and various indicators of suspicious patterns. For example, the presence of keywords such as *login*, *secure*, or *bank* may suggest malicious intent. In addition, the domain name is analyzed to identify unusual characters or uncommon top-level domains.

The feature extraction process is implemented in Python, as shown in the following example:

```
1 def url_format(url):
2     parsed_url = urlsplit(url)
3
4     return {
5         'url_length': len(url),
6         'num_dots': url.count('.'),
7         'num_slash': url.count('/'),
8         'https_protocol': int(parsed_url.scheme == "https"),
9         'num_special_char_domain': str(parsed_url.netloc).count(r'
10             [^A-Za-z0-9-]'),
11         'suspicious_word_domain': int(any(word in parsed_url.netloc
12             .lower() for word in suspicious_words)),
13         'top_tld': int(parsed_url.netloc.split(".")[1] in top_tlds
14             ),
15         'len_path': len(parsed_url.path),
16         'suspicious_char_path': int(any(word in parsed_url.path.
17             lower() for word in suspicious_words))
18     }
```


To prepare the full dataset for training, another function, `get_data_trainable()`, was implemented. This function removes unnecessary columns, handles missing values, and applies the feature extraction process to all URLs.

```
1 def get_data_trainable(data, url_column="url", columns=None,
2   one_hot=None):
3     data = data.dropna()
4
5     if columns is not None:
6         data = data.drop(columns=columns)
7
8     if data[url_column] is not None:
9         urls_data = data[url_column].apply(url_format).apply(pd.
10            Series)
11
12     if one_hot is not None:
13         data = pd.get_dummies(data, columns=one_hot, drop_first=
14            True)
15
16     data = data.drop(url_column, axis='columns')
17
18     df_final = pd.concat([data, urls_data], axis=1)
19
20     return df_final
```

This structured representation allows the model to identify general patterns in phishing URLs, especially those related to length, structure, and keyword usage. However, this approach depends on manually defined features, which may not capture more complex relationships.

The second approach takes a different direction by keeping the raw URL as text. Instead of extracting predefined features, the URL is used directly as input for a transformer-based model. In this case, each sample consists of the original URL and a label indicating whether it is *Safe* or *Threat*.

A custom function called `create_final_nlp_dataset()` was used to convert all datasets into a common binary classification format.

```
1 def create_final_nlp_dataset(dataset, detection_cols=None,
2   main_cols=None):
3     main_data = dataset[main_cols].copy()
4     feature_change = dataset[detection_cols].copy()
5
6     feature_change["is_threat"] = feature_change[detection_cols].
7       any(axis=1) \
8       .map({True: 1, False: 0})
9
10    feature_change = feature_change["is_threat"]
```

```
11     final_dataset = pd.concat([main_data, feature_change], axis=1)
12
13     return final_dataset
```

This dataset is used to train the transformer-based model (URLBERT), which learns patterns directly from the text.

The use of both structured and raw datasets allows a comparison between traditional Machine Learning methods and modern NLP-based approaches. This combination improves the flexibility of the system and provides a better understanding of how different techniques perform in phishing detection tasks.

3.5 Advantages of the Proposed Solution

The proposed solution offers several important advantages compared to traditional phishing detection methods and existing systems. One of the main advantages is the ability to perform real-time detection. Unlike many systems that analyze data after the user interacts with a website, this solution intercepts requests before they are executed. This reduces the risk of exposure to phishing attacks and provides an additional layer of protection.

Another important advantage is the use of a transformer-based Machine Learning model. The system is able to analyze URLs as text and detect complex patterns that are difficult to capture using simple rules. This allows the system to handle more advanced phishing techniques.

From a usability perspective, the application is designed to be simple and accessible. The user interface built with .NET MAUI does not require technical knowledge, which makes the solution suitable for a wide range of users. At the same time, the system keeps the user involved by allowing them to override decisions when necessary.

Another advantage is the use of a proxy-based worker that continuously monitors traffic in the background. This ensures that detection is always active, even when the user interface is not open. In addition, the use of SignalR enables real-time communication between components, providing immediate feedback to the user. The system also maintains user privacy by storing data locally using SQLite, reducing the need for external data transmission. This makes the solution more secure and easier to deploy.

Finally, the architecture is modular, which allows future improvements. New models, datasets, or features can be added without redesigning the entire system. This makes the solution scalable and adaptable to new types of phishing attacks.

3.6 Comparison with Existing Approaches

The proposed system was designed by taking into account the limitations of existing phishing detection methods. In order to better understand its contribution, it is important to compare it with the main approaches identified in the literature.

One of the most common detection techniques is based on rule-based systems, such as blacklists. These systems rely on previously identified malicious URLs and block access when a match is found. As described by Alanezi, this approach is simple and efficient, making it suitable for large-scale environments [3]. However, it has a major limitation: it can only detect known threats. New phishing websites that are not yet included in the blacklist can easily bypass detection.

Machine Learning models were introduced to improve detection by learning patterns from data instead of using fixed rules. Algorithms such as Random Forest and Support Vector Machines have shown good performance when using structured features [6]. These models can detect previously unseen threats, which represents an important advantage over rule-based systems. At the same time, they depend heavily on feature engineering. If important features are missing or not well defined, the model performance may be affected.

More recent approaches use deep learning and NLP-based models, which can learn directly from raw data. According to Catal et al., models such as neural networks and recurrent architectures are able to capture more complex patterns in phishing data [9]. These models are more flexible and do not require manual feature extraction. However, most of these solutions are evaluated in offline scenarios and are not always integrated into real systems.

The proposed solution combines elements from all these approaches and extends them. Similar to classical Machine Learning systems, it supports structured feature extraction. At the same time, it uses a transformer-based model (URLBERT), which works directly with raw URL data. This allows the system to detect more complex phishing patterns without relying only on predefined features.

From a usability perspective, the system also offers improvements. Unlike many technical solutions, this application is designed to be used by non-technical users. The interface is simple, and the system provides clear notifications, while still allowing the user to make the final decision. This makes it not only effective in detecting phishing attacks, but also practical for real-world use.

Chapter 4

System Implementation and Evaluation

This chapter presents the implementation details of the developed application and explains how the main components are integrated into a functional system. The architecture described previously is translated into a working application that includes a frontend interface, backend services, a worker component, and a Machine Learning model used for phishing detection. The goal of this chapter is to show that the solution is not only conceptually valid, but also practically implemented and capable of handling real-world scenarios.

4.1 Application Architecture

Phishing attacks are designed to deceive users into providing sensitive information through carefully crafted messages and fake websites. Even with existing protection mechanisms and user awareness, attackers continue to develop new techniques, making detection more challenging. As highlighted in the literature, traditional approaches often suffer from high false positive and false negative rates, which limits their effectiveness in real-world scenarios [14].

This section presents the application architecture of the proposed system. The architecture is designed to separate responsibilities across multiple layers, including controllers, services, data access components, and supporting modules. This layered approach improves the maintainability and scalability of the application, while also allowing easy integration of Machine Learning components. Figure 4.1 illustrates the overall structure of the application, including the main layers and the flow of data between them. The architecture follows a modular design, where each component has a well-defined role, ensuring that the system remains flexible and easy to extend in the future.

The application architecture is designed using a layered approach, where each component has a specific role in the system. This separation allows better organization of the application and simplifies future development and maintenance. Architecture is the flow

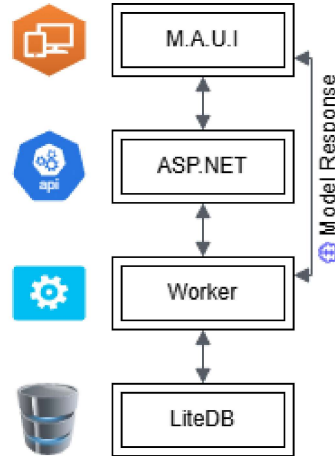


Figure 4.1: Application architecture of the proposed phishing detection system. The diagram illustrates the main components, including the MAUI frontend, ASP.NET backend, worker component, and the local database (LiteDB), as well as the flow of data between them.

of data between components. When a user interacts with the application, requests are sent from the frontend to the backend. The backend processes these requests and communicates with the worker component, which performs the actual analysis using the Machine Learning model. The results are then stored in the database and sent back to the user interface. This approach ensures that the system can operate in real time, while maintaining a clear separation between the user interface, business logic, and data processing layers. The architectural design also allows flexibility, as new components or models can be added without affecting the entire system. This makes the application scalable and suitable for future improvements.

4.2 Database Design

When designing a database system, developers often face the decision between using a database-dependent solution or a more flexible, database-independent approach. In practice, this decision is influenced by several factors such as development time, available resources, and system requirements [15]. In the context of this application, a database-dependent approach was chosen by using LiteDB, which is optimized for local applications. This decision simplifies the system architecture and reduces complexity, making it more suitable for a lightweight, real-time phishing detection system.

The database is designed to support the main functionality of the system, including user management, device monitoring, threat detection, and alert generation. Figure 4.2 presents the main entities used in the database and the relationships between them.

The data model consists of four main entities: *User*, *Device*, *ThreatEvent*, and *Alert*.

The **User** entity represents the application user and stores basic information required for system interaction.

The **Device** entity is associated with a user and represents a monitored device. In the

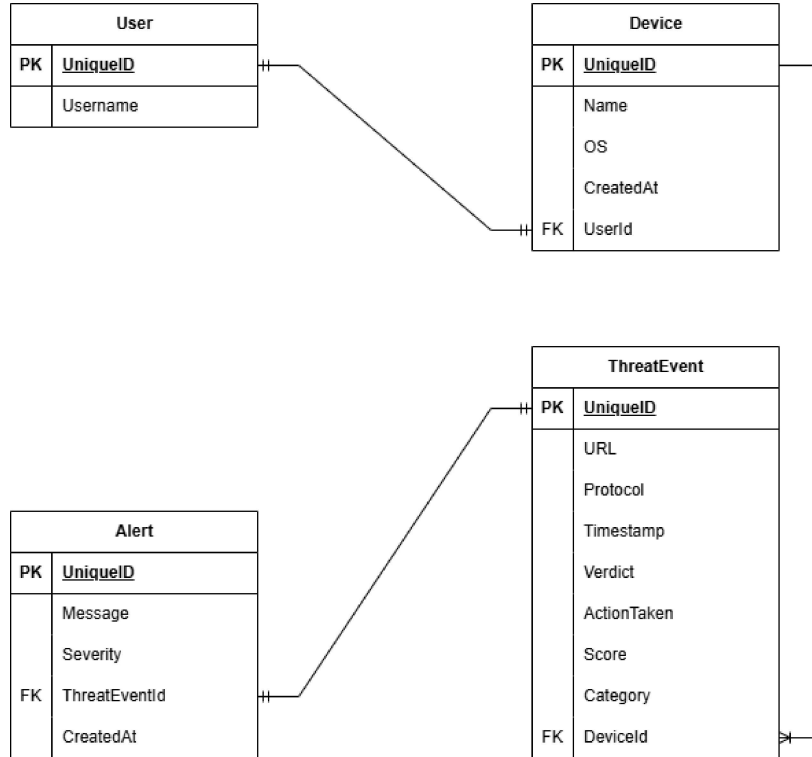


Figure 4.2: Database structure showing the relationships between User, Device, ThreatEvent, and Alert entities.

current implementation, each user is associated with a single device, as the system is designed to operate in a local environment. However, the data model supports a one-to-many relationship between users and devices, allowing a user to manage multiple devices if needed.

The **ThreatEvent** entity stores information about detected threats. It acts as a bridge between the detection process and alert generation. Each threat event is linked to a specific device and contains details about the detected phishing activity.

The **Alert** entity represents notifications generated by the system. Alerts are created based on detected threat events and are used to inform the user about potential risks.

Data flow follows a hierarchical structure, where user actions lead to device activity, which may generate threat events. These events are then processed and transformed into alerts that are displayed to the user. This simple and modular database design ensures flexibility and allows the system to be easily extended in the future. Additional entities or relationships can be added without major changes to the existing structure.

4.3 ML Model and Training Process

As described in the previous sections, the dataset used in this work was constructed by combining multiple sources and applying preprocessing techniques. After cleaning the data and removing incomplete entries, a total of approximately 800,000 valid samples were obtained.

This dataset provided a solid foundation for training both classical Machine Learning models and modern NLP-based approaches. One important aspect when working with large datasets is the balance between classes. In phishing detection, it is common for legitimate samples to be more frequent than malicious ones, which may affect model performance. Previous studies have shown that class imbalance can influence the learning process, especially when the minority class contains fewer representative examples [16]. For this reason, care was taken to use representative subsets of the dataset during training and evaluation.

4.3.1 Random Forest Model

The first model used in this work is a classical Machine Learning approach based on the Random Forest algorithm [17]. This model was trained using structured features extracted from URLs, as described in the Feature Extraction section. Due to computational constraints and the need for efficient experimentation, a subset of 40,000 samples was selected from the full dataset. The model was trained and evaluated using this subset, achieving moderate performance.

The evaluation results are presented using standard metrics such as precision, recall, F1-score, and accuracy. As shown in Figure 4.3, the model achieved an overall accuracy of approximately 0.64, with similar values for precision and recall. Although the results are acceptable for a baseline approach, they indicate that the model has limitations in capturing more complex phishing patterns. The confusion matrix further highlights the model's difficulty in distinguishing between safe and malicious URLs, particularly in borderline cases where structural features are not sufficient.

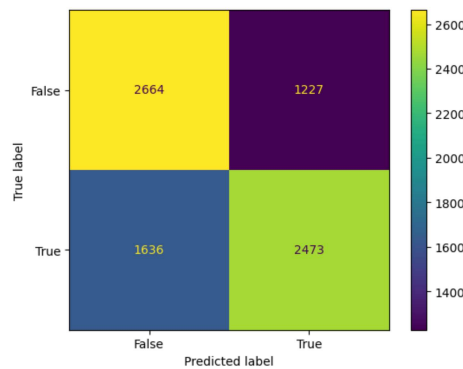


Figure 4.3: Confusion matrix and classification report for the Random Forest model.

4.3.2 NLP-Based Model (URLBERT)

In order to improve detection performance, a transformer-based model was used [18]. Unlike classical approaches, this model operates directly on raw URL text and is able to learn more complex patterns. The NLP model was initially trained on a subset of 30,000 samples

4.4. The performance obtained at this stage was significantly better than the Random Forest model, confirming the advantages of deep learning approaches in this context. To further improve the results, the training dataset was extended to 40,000 samples 4.5. The model was trained for multiple epochs, and its performance was evaluated after each iteration.

The model improves consistently across epochs. Both training and validation loss decrease over time, while accuracy and F1-score increase.

 [14070/14070 04:10, Epoch 15/15]

Epoch	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall
1	0.597819	0.511512	0.723167	0.670371	0.811239	0.571187
2	0.526908	0.494956	0.649000	0.691925	0.609693	0.799797
3	0.517214	0.481718	0.728167	0.691624	0.784305	0.618532
4	0.504379	0.466544	0.753000	0.702290	0.864918	0.591140
5	0.494443	0.457550	0.642833	0.710366	0.591625	0.888739
6	0.485486	0.436457	0.743167	0.733897	0.749824	0.718634
7	0.471001	0.421656	0.740000	0.746589	0.718349	0.777139
8	0.458221	0.410162	0.774000	0.760170	0.796811	0.726750
9	0.435364	0.385350	0.761167	0.779029	0.715986	0.854244
10	0.420585	0.362510	0.779833	0.793626	0.737515	0.858979
11	0.404066	0.341279	0.840000	0.827089	0.884778	0.776463
12	0.382979	0.327365	0.807667	0.821418	0.757204	0.897531
13	0.376689	0.303787	0.833667	0.838511	0.803909	0.876226
14	0.364782	0.296881	0.843333	0.844936	0.824799	0.866080
15	0.348887	0.287461	0.849167	0.849342	0.836393	0.862699

Figure 4.4: Training process of the NLP-based phishing detection model with 15 Epoch.

The final model achieved an accuracy of over 0.90 and an F1-score above 0.91, which represents a significant improvement compared to the Random Forest model. The balance between precision and recall indicates that the model is able to detect phishing attempts reliably, without generating a large number of false positives.

 [31250/31250 03:36, Epoch 25/25]

Epoch	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall
16	0.401176	0.295287	0.868857	0.868179	0.855889	0.880828
17	0.386178	0.287921	0.878571	0.871212	0.907513	0.837704
18	0.381377	0.274114	0.879143	0.878553	0.865874	0.891608
19	0.364909	0.249242	0.878857	0.882938	0.838930	0.931818
20	0.354983	0.235673	0.900714	0.899377	0.893813	0.905012
21	0.335386	0.213760	0.893714	0.896753	0.856121	0.941434
22	0.318772	0.207048	0.913286	0.912067	0.906943	0.917249
23	0.312837	0.192319	0.908143	0.909500	0.879662	0.941434
24	0.303119	0.188479	0.910286	0.911499	0.882642	0.942308
25	0.290940	0.184531	0.914571	0.915465	0.889072	0.943473

Figure 4.5: Training process of the NLP-based phishing detection model with 25 Epoch.

4.3.3 Training Observations

Transformer-based models were able to learn useful patterns more efficiently than classical approaches. Even with a relatively small subset of the dataset, the NLP model achieved high performance. The results highlight the importance of data quality rather than only data quantity. A well-prepared subset of 30,000–40,000 samples was sufficient to train an effective model. The results align with previous research, which shows that Machine Learning performance depends not only on the dataset size, but also on preprocessing, feature representation, and model architecture [19].

4.3.4 Backend Architecture and Bussines Logic

This chapter presents the backend architecture and the core business logic of the application. The system is designed using a modular, service-oriented approach, combining multiple technologies to ensure scalability, real-time processing, and efficient threat detection. The architecture consists of four main components: a Python-based model server and a background worker acting as a proxy, an ASP.NET backend server implementing business logic.

4.4 Python Model Server

The Python server represents a core component of the application, being responsible for detecting potential threats. It is implemented using the Flask framework, which enables the creation of lightweight and efficient REST APIs for communication with other components.

The machine learning model is initialized using the PyTorch library, where trained weights are loaded and prepared for inference. The model operates in the Natural Language Processing domain and has been trained specifically to classify URLs as either safe or malicious.

To improve prediction reliability and reduce the number of false positives, an additional filtering mechanism is introduced before model inference. This mechanism relies on the Tranco list, a research-oriented ranking of the most popular and trustworthy domains [20]. By checking whether a given URL contains elements from this list, the system can identify known safe domains. This step is necessary because NLP models trained on specific datasets may interpret unfamiliar but legitimate patterns as threats if such patterns are not represented in the training data.

After the filtering stage, the input URL is processed through a normalization pipeline. This includes parsing the URL, cleaning the input, tokenizing it, and transforming the data into a machine-readable format suitable for the model. The processed data is then passed to the model, which produces a classification label indicating whether the URL is safe or represents a threat, along with a confidence score associated with the prediction.

The Python server exposes an API endpoint, such as `/model_predict`, which allows external components to request real-time predictions. This endpoint is used primarily by the worker component to evaluate URLs dynamically.

The interaction between the worker component and the machine learning model is achieved through a REST API implemented in the Python server. Listing 4.1 presents the endpoint responsible for handling real-time URL classification requests.

Listing 4.1: Model prediction API endpoint

```
1 @app.route('/model_predict', methods=["POST"])
2 def uselmodel():
3     data = request.get_json()
4     url = data.get("url")
5
6     response = filter_urls(url)
7     if(response == 'false'):
8         response = predict_sweeb(url)
9
10    return jsonify({
11        "url": url,
12        "prediction": response
13    })
```

4.5 Worker Component

The worker component functions as an intermediary layer between user activity and backend processing. Its role is to act as a proxy that intercepts and monitors user requests in real time.

The worker operates continuously in the background and is automatically initialized when the application starts, without requiring manual configuration. It captures live traffic data using SignalR, enabling real-time communication with the backend infrastructure.

When a user initiates a request, the worker analyzes it and determines whether it may be suspicious. If necessary, the URL associated with the request is forwarded to the Python model server via the `/model_predict` endpoint. Based on the response received, the worker decides how to proceed. If the URL is classified as safe, the request is allowed to continue without interruption. In contrast, if the model identifies the URL as a threat, access to the corresponding webpage is blocked.

In the case of a detected threat, the worker also triggers a notification within the application. The user is informed about the potential risk and is given the opportunity to make a final decision, either maintaining the block or allowing access despite the warning. This approach ensures both automated protection and user control over browsing decisions.

The worker component is implemented as a background service responsible for continuously monitoring and intercepting user traffic. Listing 4.2 illustrates the core execution logic of the worker.

Listing 4.2: Background worker execution logic

```
1 public class ProxyWorker(IProxyService proxy, ISignalRService
   signalRService) : BackgroundService
2 {
3     private readonly IProxyService _proxyService = proxy;
4     private readonly ISignalRService _signalRService =
       signalRService;
5
6     protected override async Task ExecuteAsync(CancellationToken
       cancellationTokentoken)
7     {
8         await _signalRService.StartAsync();
9         await _proxyService.StartAsync(cancellationTokentoken);
10
11         try
12         {
13             await Task.Delay(Timeout.Infinite, cancellationTokentoken);
14         }
15         catch (TaskCanceledException) {}
16
17         await _proxyService.StopAsync(cancellationTokentoken);
18         await _signalRService.StopAsync();
19     }
20 }
```

4.6 ASP.NET Backend Server

The ASP.NET backend serves as the central component responsible for implementing business logic and managing communication between all system elements. It handles data processing, persistence, and real-time data transmission to the frontend.

The backend is responsible for performing all CRUD operations, managing database interactions using SQLite, and ensuring that data is stored and transferred securely. It also plays a key role in relaying live data updates to the frontend, allowing users to observe system activity in real time.

The application follows a layered architectural pattern structured as Models, Repositories, Services, and Controllers. This design separates concerns and improves maintainability. Models define the domain entities used throughout the system, repositories handle direct interaction with the database, services encapsulate the business logic, and controllers expose API endpoints to external clients.

The project structure is organized into key components, including Controllers, Data repositories, Models, Enums, and Services. This organization ensures clarity in development and allows for easier extension of functionality over time.

Real-time communication between the backend and the frontend is implemented using SignalR. Listing 4.3 shows the hub responsible for broadcasting intercepted traffic data to all connected clients.

Listing 4.3: SignalR Hub for real-time traffic communication

```
1 public class SignalRHub : Hub
2 {
3     public async Task SendProxyTrafficAsync(ResponseProxy response)
4     {
5         Console.WriteLine($"[API RECEIVED] {response.Url}");
6
7         await Clients.All.SendAsync("ReceiveProxyTraffic", response
8             );
9     }
10 }
11
12 public static class MiddlewareExtensions
13 {
14     public static WebApplication UseAppPipeline(this
15         WebApplication app)
16     {
17         app.MapHub<SignalRHub>("/signalRHub");
18     }
19 }
```

4.7 Frontend Implementation

The frontend of the application was developed using .NET MAUI, a cross-platform framework that enables the creation of applications for multiple operating systems, including desktop and mobile environments. This choice ensures scalability, maintainability, and future extensibility of the system. Additionally, it allows the application to be distributed across different platforms without requiring major architectural changes.

The frontend communicates with the backend services described in the previous chapter and provides an intuitive interface for end users. The design prioritizes usability, allowing non-technical users to interact efficiently with the system.

MVVM Architectural Pattern

The frontend follows the Model-View-ViewModel (MVVM) architectural pattern, as recommended in the official Microsoft documentation [12]. This pattern separates application logic from the user interface, improving maintainability, testability, and scalability.

According to Microsoft, MVVM consists of three main components:

- ▷ The **Model**, which represents the data and business logic
- ▷ The **View**, which defines the graphical user interface
- ▷ The **ViewModel**, which acts as an intermediary between the Model and the View, exposing data through properties and handling user interactions

The ViewModel uses data binding to ensure that changes in the UI are automatically reflected in the underlying data and vice versa. Commands are used to handle user actions, eliminating the need for code-behind logic and ensuring a clean separation of concerns.

Frontend Architecture

The project structure follows a modular organization aligned with the MVVM pattern:

- ▷ **Views** – XAML pages representing the user interface
- ▷ **ViewModels** – Classes handling UI logic and state
- ▷ **Models** – Data structures exchanged with the backend
- ▷ **Services** – Classes responsible for communication and navigation
- ▷ **Components** – Reusable UI elements

This structure allows for clear separation between presentation and logic, making the application easier to extend and maintain.

Navigation Mechanism Navigation between pages is implemented using a dedicated service that abstracts view resolution. This approach decouples the navigation logic from the ViewModel and improves flexibility.

Listing 4.4 presents the implementation of the navigation service.

Listing 4.4: Navigation service implementation

```
1 public class NavigationService : INavigationService
2 {
3     private readonly IServiceProvider _provider;
4
5     public NavigationService(IServiceProvider provider)
6     {
7         _provider = provider;
8     }
9
10    public View GetView(string key)
11    {
12        return key switch
13        {
14            "Dashboard" => _provider.GetRequiredService<
15                DashboardView>(),
16            "LiveRequests" => _provider.GetRequiredService<
17                LiveRequestsView>(),
18            "Threats" => _provider.GetRequiredService<ThreatsView>(),
19            _ => _provider.GetRequiredService<DashboardView>()
20        };
21    }
22 }
```

As shown in Listing 4.4, the service receives a key and returns the corresponding view using dependency injection. This allows for centralized navigation management and simplifies future extensions.

The interface used by the service is defined in Listing 4.5.

Listing 4.5: Navigation service interface

```
1 public interface INavigationService
2 {
3     View GetView(string key);
4 }
```

ViewModel Implementation The ViewModel handles navigation logic and maintains the current state of the interface. It exposes properties bound to the UI and commands triggered by user interactions.

Listing 4.6 illustrates the implementation of the main ViewModel.

Listing 4.6: Main ViewModel handling UI logic and navigation

```
1 public class MainPageViewModel : BaseViewModel
2 {
3     private readonly INavigationService _navigationService;
4
5     private View? _currentView;
6     private string _currentPageKey = "Dashboard";
7
8     public View? CurrentView
9     {
10         get => _currentView;
11         set => SetProperty(ref _currentView, value);
12     }
13
14     public string CurrentPageKey
15     {
16         get => _currentPageKey;
17         set => SetProperty(ref _currentPageKey, value);
18     }
19
20     public ICommand NavigateCommand { get; }
21
22     public MainPageViewModel(INavigationService navigationService)
23     {
24         _navigationService = navigationService;
25
26         NavigateCommand = new Command<string>(Navigate);
27         Navigate(CurrentPageKey);
28     }
29
30     private void Navigate(string key)
31     {
32         CurrentPageKey = key;
33         CurrentView = _navigationService.GetView(key);
34     }
35 }
```

The ViewModel shown in Listing 4.6 demonstrates how navigation is handled through commands. When a user interacts with the interface, the command is triggered, and the corresponding view is loaded dynamically.

User Interface Components The user interface is defined using XAML and relies on reusable components to ensure consistency and modularity. One important component is the sidebar, which provides navigation between application sections.

Listing 4.7 presents part of the sidebar implementation.

Listing 4.7: Sidebar component with navigation bindings

```
1 <ContentView xmlns="http://schemas.microsoft.com/dotnet/2021/maui"  
2     xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"  
3     x:Class="SweebAppFront.Components.SidebarView"  
4     xmlns:components="clr-namespace:SweebAppFront.  
5         Components"  
6     Background="{DynamicResource Accent}">  
7  
8     <Border Stroke="{DynamicResource Variant}" StrokeThickness="3"  
9         Padding="0">  
10         <VerticalStackLayout Spacing="20" Margin="20">  
11             <components:MenuButton  
12                 IconPath="dashboard.png"  
13                 MenuText="Dashboard"  
14                 PageKey="Dashboard"  
15                 CurrentPageKey="{Binding CurrentPageKey}"  
16                 Command="{Binding NavigateCommand}"  
17                 CommandParam="Dashboard"  
18             />  
19         />  
20     />
```

The component shown in Listing 4.7 demonstrates how data binding is used to connect the user interface to the ViewModel. The navigation command is triggered when the user interacts with the menu, ensuring a responsive and dynamic interface.

In the context of the developed application, the frontend is directly connected to backend services that provide real-time data regarding user requests and detected threats. The MVVM architecture ensures that these updates are reflected instantly in the interface.

For example, when the worker component detects a potential threat and sends data through the backend, the frontend receives this information via SignalR. The ViewModel updates its properties accordingly, and the UI automatically reflects the changes without requiring manual refresh.

The Dashboard displays aggregated threat statistics, the Live Requests page shows real-time user activity, and the Threats page allows users to manage and filter detected threats. This dynamic behavior is made possible by the combination of MVVM data binding and service-based architecture.

The frontend implementation leverages the capabilities of .NET MAUI and the MVVM pattern to create a scalable and maintainable application. By separating concerns between UI, logic, and services, the system ensures flexibility and ease of development.

The use of navigation services, data binding, and reusable components results in a clean architecture that integrates seamlessly with the backend. Following Microsoft's recommended practices [12], the application achieves both robustness and usability, making it suitable for further expansion and cross-platform deployment.

4.8 User Interaction and System Workflow

The SweebApp prototype was designed as an interactive real-time phishing detection system. The user interacts with the .NET MAUI frontend, while the ASP.NET Core backend, the C# worker component, the SQLite database, and the Python/Flask model server operate locally in the background.

All API endpoints and ports used in this implementation are exposed only for local testing, development, and local inter-process communication. They are not public production APIs and are not intended to be exposed over the Internet. For example, the backend runs locally on addresses such as `https://localhost:7832`, while the model server exposes a local endpoint such as `http://127.0.0.1:7833/model_predict`.

On first launch, the user enters a username. The MAUI frontend sends this value to the local backend, where the user is created or retrieved. The application then stores the local authentication state using an authentication state service. After this step, the current device information, including device name and operating system, is registered and displayed in the Dashboard.

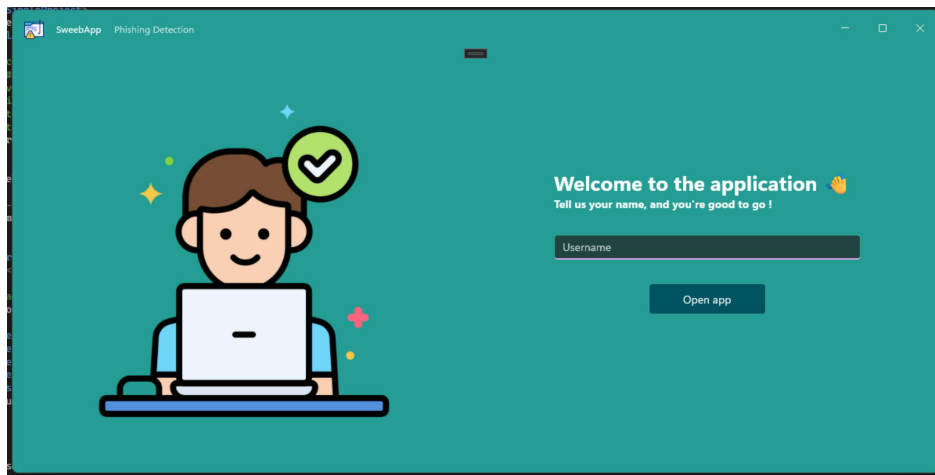


Figure 4.6: Login interface used to initialize the local user profile.

After initialization, the API is started first, followed by the worker component. The worker connects to the local SignalR hub and starts the local proxy. During execution, the worker configures the Windows proxy to `127.0.0.1:8000`, allowing outgoing browser requests to be intercepted and analyzed before the user reaches a potentially dangerous page.

The detection workflow uses a hybrid approach based on whitelist filtering and NLP-based classification. The whitelist was introduced because the NLP model alone cannot reliably know all legitimate domains, since the Internet contains a very large number of safe websites. Therefore, before a URL is analyzed by the NLP model, it is compared against a trusted-domain whitelist. If the URL matches a trusted entry, the request is considered safe and is allowed to continue. If no whitelist match is found, the URL is forwarded to the local NLP-based prediction endpoint for further classification.

This approach reduces false positives for known legitimate domains while still allowing unknown or suspicious URLs to be analyzed by the trained model. Static resources such as JavaScript files, stylesheets, images, assets, socket connections, and polling requests may also be ignored to reduce noise in the traffic stream.

If the URL is classified as safe, the request continues without interruption. If the URL is classified as a threat, the worker blocks the request, creates a `ThreatEvent`, and generates an `Alert`. These records are stored in the local SQLite database through Entity Framework Core. The backend then broadcasts the new threat and alert data through SignalR, allowing the frontend to update the Live Requests, Threats, Alerts, and Dashboard views in real time.

Detected threats are displayed in the Threats page. Each threat can have the status `Blocked` or `Allowed`. By default, detected phishing threats are blocked. If the user manually allows a blocked threat, the system generates a warning alert explaining that the user is responsible for the potential risk. Future requests to an allowed threat are allowed to continue, while blocked threats remain prevented by the worker.

The Dashboard summarizes the system state by showing the total number of threats, blocked threats, allowed threats, and a monthly aggregation across all twelve months of the current year. It also displays the current device name and operating system.

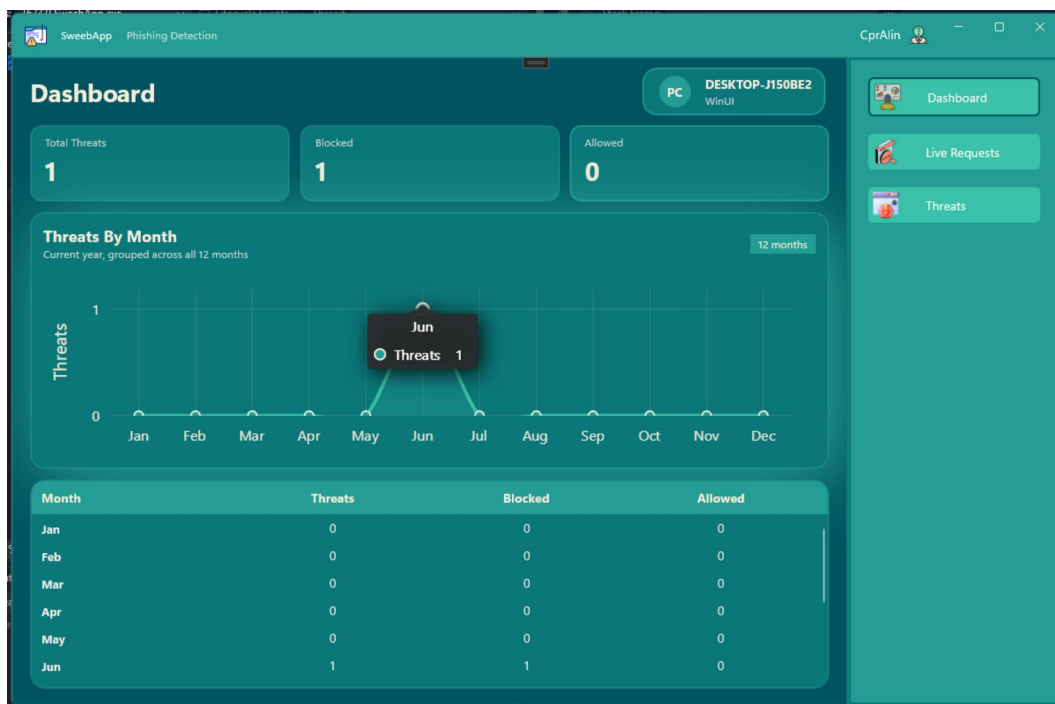


Figure 4.7: Dashboard showing aggregated threat statistics, monthly threat distribution, and current device information.

The Live Requests page displays intercepted traffic received through SignalR. The Alerts view shows generated security notifications and allows the user to close individual alerts or clear all alerts.

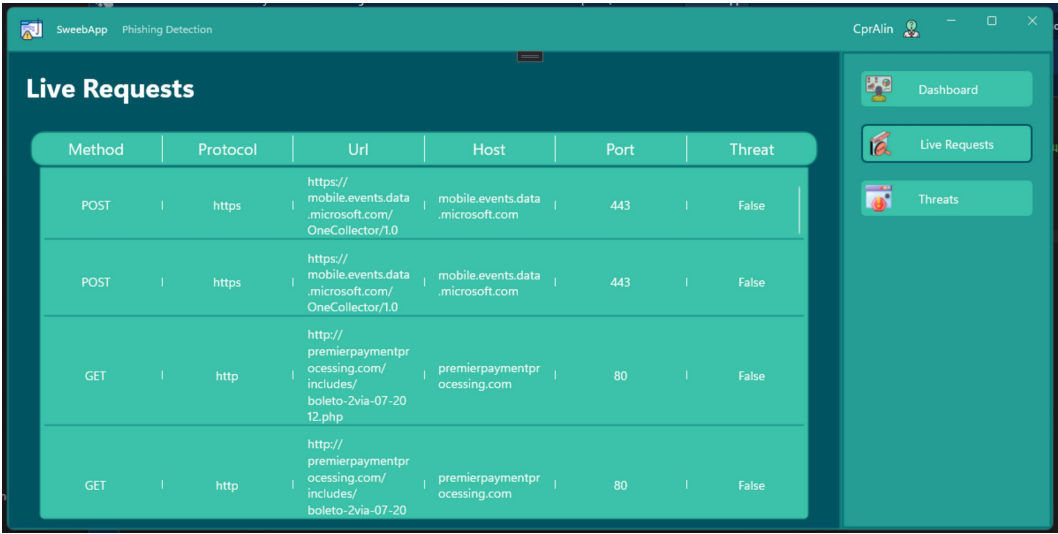


Figure 4.8: Live Requests page displaying intercepted traffic in real time.

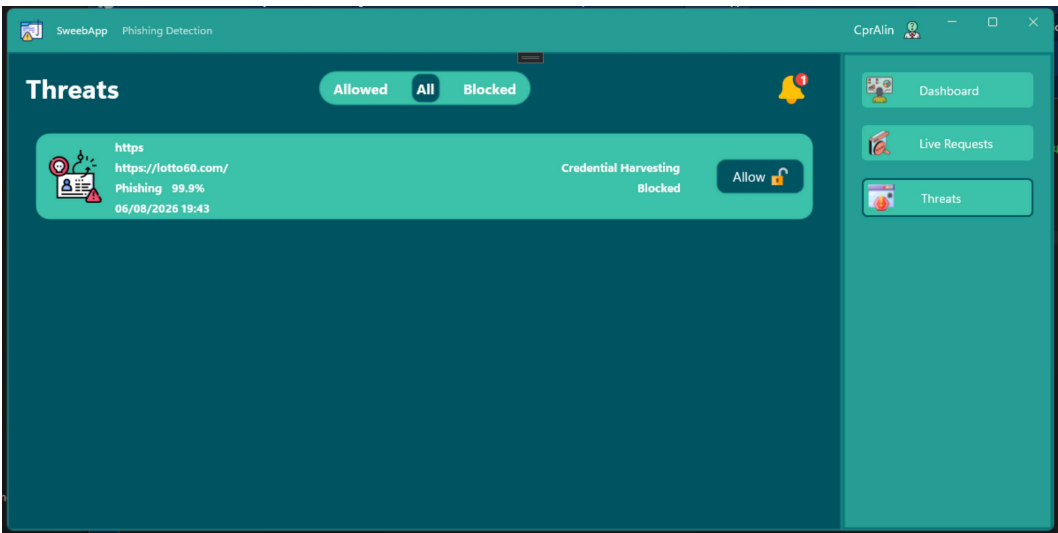


Figure 4.9: Threat management page with filtering and allow/block actions.

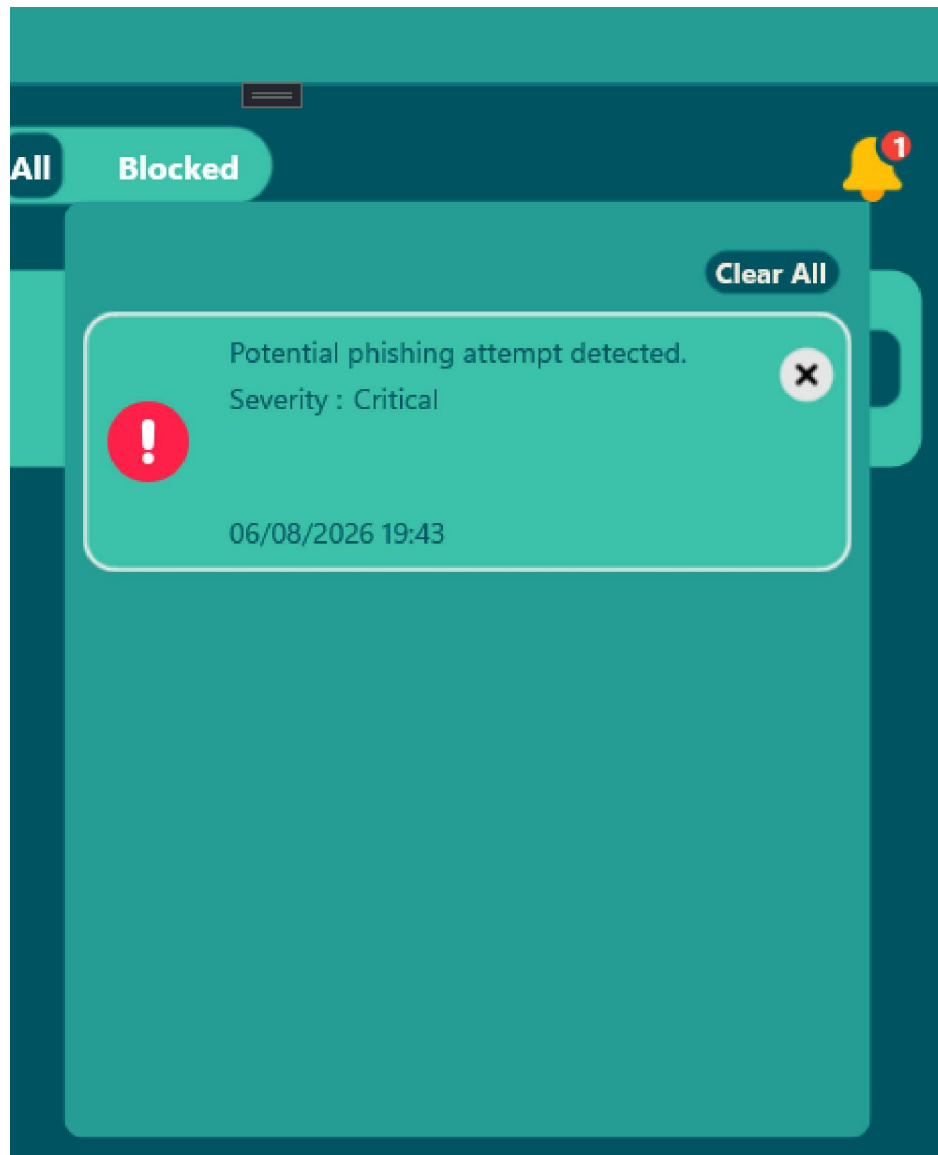


Figure 4.10: Alert popup showing generated security notifications and alert management actions.

When a dangerous request is blocked, the worker returns a local block page. This informs the user that the request was prevented because it was classified as a potential phishing threat.

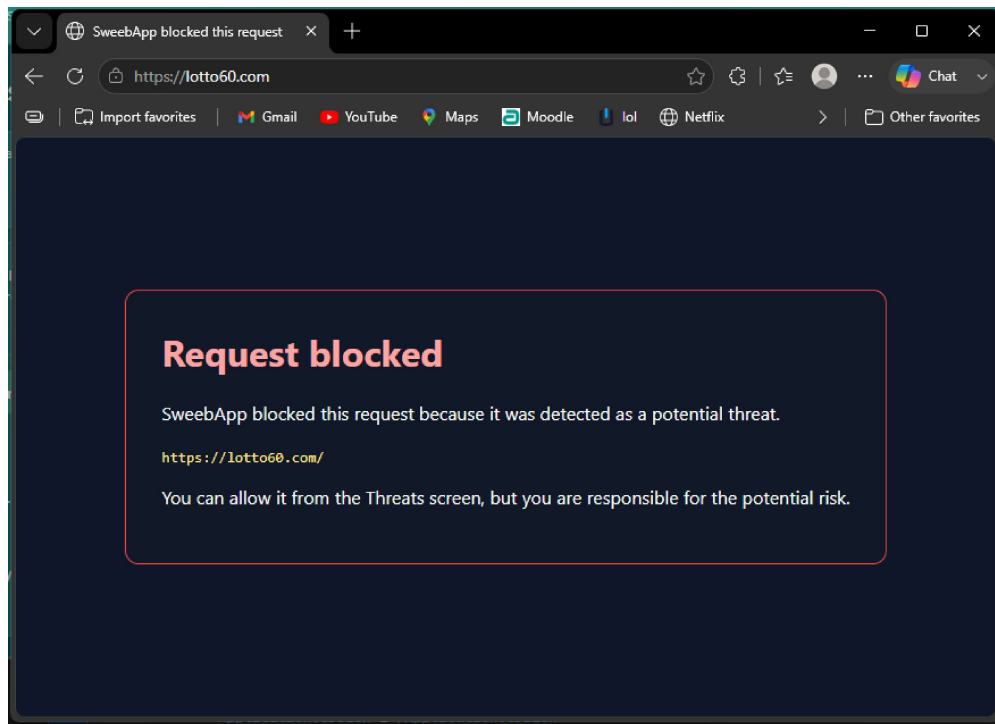


Figure 4.11: Local block page displayed when a detected phishing request is prevented.

The experimental setup was built in a local Windows environment. The system consists of a .NET MAUI frontend, an ASP.NET Core backend, a C# worker service, a SQLite database accessed through Entity Framework Core, a Python/Flask model server, SignalR for real-time communication, and Titanium.Web.Proxy for local traffic interception.

The backend API, SignalR hub, worker proxy, and model prediction endpoint are used only locally. They are part of the prototype and testing architecture and are not public Internet-facing services. The local HTTPS backend also requires certificate configuration when the application is deployed on another device.

The main tested scenarios were: safe URL access, phishing URL access, whitelist-based safe domain handling, NLP-based classification for unknown URLs, threat creation, alert generation, live request streaming, allow/block actions, individual alert deletion, clear all alerts, and Dashboard monthly aggregation.

```

2 references
public async Task StartAsync(CancellationTok... cancellationToken)
{
    SetSystemProxy("127.0.0.1:8000");
    _proxyServer.CertificateManager.EnsureRootCertificate();

    var explicitEndPoint = new ExplicitProxyEndPoint(IPAddress.Any, 8000, true);
    _proxyServer.AddEndPoint(explicitEndPoint);

    _proxyServer.BeforeRequest += OnRequest;

    _proxyServer.Start();

    Console.WriteLine("Proxy server started on port 8000.");
}

```

Figure 4.12: Local Windows proxy configuration used by the worker component during testing.

4.9 Performance Evaluation and Results

The Machine Learning model evaluation was presented in the previous sections. This section focuses on the behavior of the complete system. The evaluation verified whether browser requests were intercepted correctly, whether whitelist filtering reduced unnecessary model calls for trusted domains, whether unknown URLs were classified by the NLP model, and whether threats were blocked and displayed correctly in the interface.

Table 4.1: Functional evaluation scenarios for the SweebApp prototype

Scenario	Expected result	Observed result	Status
Safe whitelisted URL	Request allowed without model inference.	The request continues normally.	Passed
Unknown URL	URL sent to the NLP model.	The prediction endpoint returns a classification.	Passed
Phishing URL	Request blocked.	A local block page is displayed.	Passed
Threat creation	Threat stored in the system.	A <code>ThreatEvent</code> is saved in SQLite.	Passed
Alert generation	Alert displayed in the UI.	The frontend receives the alert through SignalR.	Passed
Live request streaming	Traffic displayed in real time.	The Live Requests page is updated.	Passed
Allow threat action	Threat marked as allowed.	The status changes and a warning alert is generated.	Passed
Block threat action	Threat remains blocked.	Future requests are prevented.	Passed
Clear alerts	Alerts removed from the interface.	The alerts view is cleared.	Passed
Dashboard aggregation	Threats grouped by month.	The monthly chart and table are updated.	Passed

Table 4.2: Qualitative system-level evaluation metrics

Metric	Description	Evaluation method
Request interception	Capturing browser requests through the local proxy.	Manual browsing tests.
Whitelist filtering	Detecting known safe domains before model inference.	Safe-domain access tests.
NLP classification	Classifying unknown URLs using the local model server.	Prediction endpoint tests.
Blocking behavior	Preventing access to dangerous URLs.	Browser access tests.
SignalR propagation	Sending live events to the MAUI frontend.	UI observation.
Persistence	Storing users, devices, threats, and alerts.	SQLite verification.
User control	Allowing and blocking detected threats.	Threat status transition tests.

Overall, the prototype demonstrates that phishing detection can be integrated with local proxy interception, whitelist-assisted NLP classification, backend persistence, and real-time user interaction.

Chapter 5

Conclusion and Future Work

5.1 Summary of the Research

This research presented SweebApp, a real-time phishing detection and prevention prototype. The system integrates a .NET MAUI frontend, an ASP.NET Core backend, a C# worker component, a SQLite database with Entity Framework Core, a Python/Flask model server, SignalR communication, and local proxy-based traffic interception.

The proposed solution combines whitelist filtering with NLP-based URL classification. The whitelist helps identify known safe domains, while the NLP model analyzes unknown URLs that require further inspection. This hybrid approach improves the practical behavior of the system by reducing false positives while preserving the ability to detect suspicious URLs.

5.2 Key Findings

The first finding is that transformer-based NLP classification performed better than the Random Forest baseline discussed earlier. This confirms that URL text can contain patterns that are better captured by sequence-based models.

The second finding is that whitelist-assisted classification is useful in practical scenarios. Since no NLP model can know every legitimate website, the whitelist provides a first validation layer for trusted domains. URLs not found in the whitelist are then analyzed by the NLP model.

The third finding is that proxy-level interception can prevent user interaction with suspicious pages before the page is reached. SignalR also improves user awareness by updating the interface immediately when live requests, alerts, or threat status changes occur.

Finally, the allow/block mechanism provides a balance between automatic protection

and user control. Threats are blocked by default, but the user can manually allow them after receiving a warning about the potential risk.

5.3 Limitations of the Proposed Approach

The current system focuses mainly on URL-based analysis and does not inspect the complete HTML content, page layout, scripts, or visual similarity of the visited website. Therefore, some phishing attacks may remain difficult to detect if the URL appears legitimate.

False positives and false negatives remain possible. The whitelist reduces false positives for known safe domains, but its effectiveness depends on the quality and coverage of the trusted-domain list. If the whitelist is incomplete, legitimate URLs may still be analyzed by the NLP model and may be incorrectly classified.

The system also depends on the availability of the local backend API and the local model server. The current endpoints are local-only and are not production-hardened for Internet exposure. The deployment is mainly Windows-oriented because the worker configures the Windows proxy. In addition, local HTTPS communication requires certificate management.

5.4 Future Research Directions

Future work could improve deployment by running the API and worker as Windows Services. This would provide better startup control, recovery, and integration with the operating system.

Another direction is centralized production configuration through files. This would remove hardcoded local endpoints and make deployment easier across different devices.

The whitelist mechanism could also be improved by integrating dynamic trusted-domain sources and threat intelligence feeds. This would help the system distinguish more accurately between known safe domains and unknown URLs requiring NLP-based analysis.

Future versions could also include HTML and content-based phishing analysis, visual similarity detection, browser extension integration, explainable AI indicators, automatic model retraining, reporting features such as PDF or CSV export, role-based users, an administrator dashboard, cloud or hybrid multi-device monitoring, and a cross-platform proxy strategy.

Bibliographic References

- [1] Muhammad Saeed Liaqat, Gohar Mumtaz, Nazish Rasheed, and Zeeshan Mubeen. Exploring phishing attacks in the ai age: A comprehensive literature review. *Journal of Cybersecurity and Business Intelligence (JCBI)*, 7(2), September 2024.
- [2] Muhammad Syafiq Kheruddin, Muhammad Adam Emir Mohd Zuber, and Muhammad Mukhlis Mohamad Radzai. Phishing attacks: Unraveling tactics, threats, and defenses in the cybersecurity landscape. *Authorea*, January 2024.
- [3] Mafaz Alanezi. Phishing detection methods: A review. *Technium*, 3(9), 2021.
- [4] Nithin Valiyaveedu, Sangeetha Jamal, Roshan Reju, Vysakh Murali, and Nithin K M. Survey and analysis on ai based phishing detection techniques. In *2021 International Conference on Communication, Control and Information Sciences (ICCISc)*, volume 1, pages 1–6, 2021.
- [5] Oladimeji Azeez Lamina, Waliu Adebayo Ayuba, Olubukola Eunice Adebisi, Gracious Ebunoluwa Michael, Ojo-Omoniyi Damilola Samuel, and Keshinro Olushola Samuel. Ai-powered phishing detection and prevention. *Path of Science*, 10(12):4001–4010, 2024.
- [6] Shakeel Ahmad, Muhammad Zaman, Ahmad Sami Al-Shamayleh, Rahiel Ahmad, Shafi’I Muhammad Abdulhamid, Ismail Ergen, and Adnan Akhunzada. Across the spectrum: In-depth review of ai-based models for phishing detection. *IEEE Open Journal of the Communications Society*, 6:2065–2089, 2024.
- [7] Hany F. Atlam and Oluwatimilehin Olayonu. Business email compromise phishing detection based on machine learning: A systematic literature review. *Electronics*, 12(1):42, 2023.
- [8] Zeinab Shahbazi, Rezvan Jalali, and Maryam Molaevand. Ai-based phishing detection and student cybersecurity awareness in the digital age. *Big Data and Cognitive Computing*, 9(8):210, 2025.

- [9] Cagatay Catal, Gorkem Giray, Bedir Tekinerdogan, Sandeep Kumar, and Shashank Shukla. Applications of deep learning for phishing detection: A systematic literature review. *Knowledge and Information Systems*, 64(6):1457–1500, 2022.
- [10] Gal Egozi and Rakesh Verma. Phishing email detection using robust nlp techniques. In *2018 IEEE International Conference on Data Mining Workshops (ICDMW)*, pages 7–12, 2018.
- [11] Wenbin Yao, Yuanhao Ding, and Xiaoyong Li. Deep learning for phishing detection. In *2018 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Ubiquitous Computing & Communications, Big Data & Cloud Computing, Social Computing & Networking, Sustainable Computing & Communications (ISPA/IUCC/BDCloud/SocialCom/SustainCom)*, pages 645–650, 2018.
- [12] Microsoft. Microsoft documentation, 2024. Accessed: 2026-05-16.
- [13] Hugging Face Inc. Hugging face documentation, 2024. Accessed: 2026-05-16.
- [14] R. Aravindhan, R. Shanmugalakshmi, K. Ramya, and Selvan C. Certain investigation on web application security: Phishing detection and phishing target discovery. In *2016 3rd International Conference on Advanced Computing and Communication Systems (ICACCS)*, volume 01, pages 1–10, 2016.
- [15] M. Karakaya and other authors. Decision support for database dependent and independent methods in software development. *Journal of Applied Sciences*, 7(1):127–131, 2007.
- [16] Gustavo E. A. P. A. Batista, Ronaldo C. Prati, and Maria Carolina Monard. A study of the behavior of several methods for balancing machine learning training data. *SIGKDD Explorations*, 6(1):20–29, 2004.
- [17] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [18] Yujie Li, Yanbin Wang, Haitao Xu, Zhenhao Guo, Zheng Cao, and Lun Zhang. Urlbert: A contrastive and adversarial pre-trained model for url classification. *arXiv e-prints*, pages arXiv–2402, 2024.
- [19] Z. Li, J. Yoon, R. Zhang, et al. Machine learning in concrete science: Applications, challenges, and best practices. *npj Computational Materials*, 8:127, 2022.
- [20] Victor Le Pochat, Tom Van Goethem, Samaneh Tajalizadehkhoob, Maciej Korczyński, and Wouter Joosen. Tranco: A research-oriented top sites ranking hardened against manipulation. In *Proceedings of the 26th Annual Network and Distributed System Security Symposium, NDSS 2019, February 2019*.